



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

September-2026

**BACHELOR OF
ENGINEERING IN**

Computer Science and Engineering

Submitted by

M Nishanth

(Reg.No.192511154)

Under the

Supervision of

Dr. T.SANGEETHA

Course: Cryptography Network Security

Course Code: CSA5115

CSA-5115 CRYPTOGRAPHY FOR NETWORK SECURITY

LAB EXPERIMENTS

EXPERIMENT 1:

CODE:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>


int main() {

    char text[100];

    int k, i;


    printf("Enter the text: ");

    fgets(text, sizeof(text), stdin);


    printf("Enter the key (1-25): ");

    scanf("%d", &k);


    if (k < 1 || k > 25) {

        printf("Invalid key! Key must be between 1 and 25.\n");

        return 1;

    }
```

```

for (i = 0; text[i] != '\0'; i++) {

    if (isupper(text[i])) {

        text[i] = ((text[i] - 'A' + k) % 26) + 'A';

    }

    else if (islower(text[i])) {

        text[i] = ((text[i] - 'a' + k) % 26) + 'a';

    }

    // Non-alphabetic characters remain unchanged

}

printf("Encrypted text: %s\n", text);

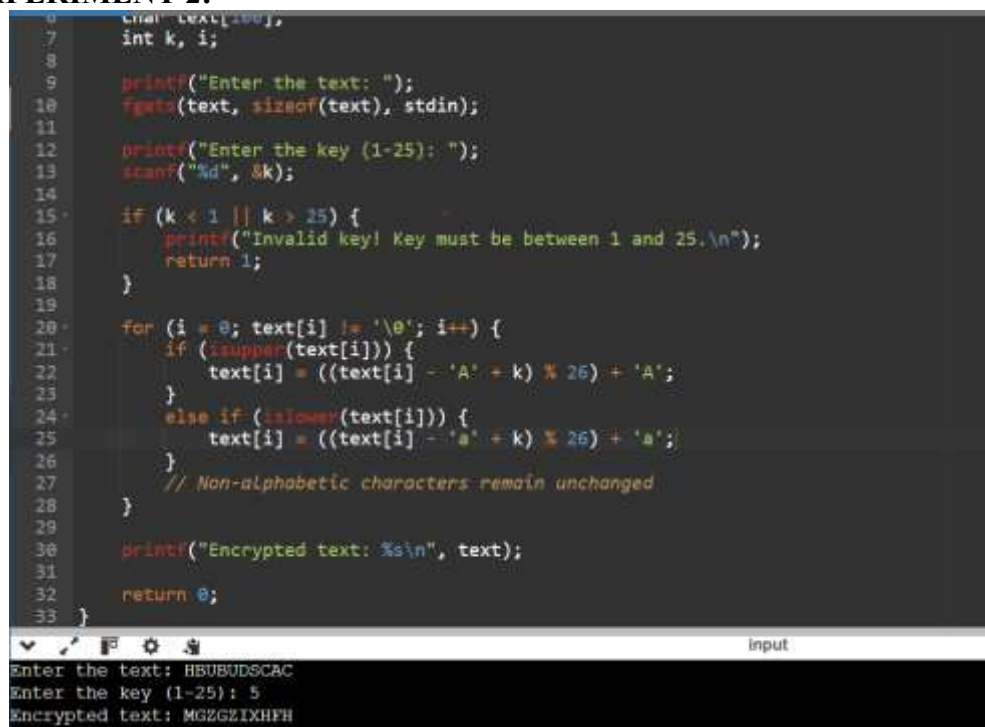
return 0;

}

```

OUTPUT:

EXPERIMENT 2:



```

1  char text[100],
2  int k, i;
3
4  printf("Enter the text: ");
5  fgets(text, sizeof(text), stdin);
6
7  printf("Enter the key (1-25): ");
8  scanf("%d", &k);
9
10 if (k < 1 || k > 25) {
11     printf("Invalid key! Key must be between 1 and 25.\n");
12     return 1;
13 }
14
15 for (i = 0; text[i] != '\0'; i++) {
16     if (isupper(text[i])) {
17         text[i] = ((text[i] - 'A' + k) % 26) + 'A';
18     }
19     else if (islower(text[i])) {
20         text[i] = ((text[i] - 'a' + k) % 26) + 'a';
21     }
22     // Non-alphabetic characters remain unchanged
23 }
24
25 printf("Encrypted text: %s\n", text);
26
27 return 0;
28 }

```

Input

```

Enter the text: HBUBUDSCAC
Enter the key (1-25): 5
Encrypted text: MGZGZIXHFI

```

CODE:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>


int main() {
    char plain[100], cipher[100];
    char key[27]; // substitution alphabet
    int i;


    printf("Enter plaintext: ");
    fgets(plain, sizeof(plain), stdin);


    printf("Enter 26-letter substitution key (e.g. QWERTYUIOPASDFGHJKLZXCVBNM):");
    scanf("%s", key);


    for(i = 0; plain[i] != '\0'; i++) {
        char ch = plain[i];

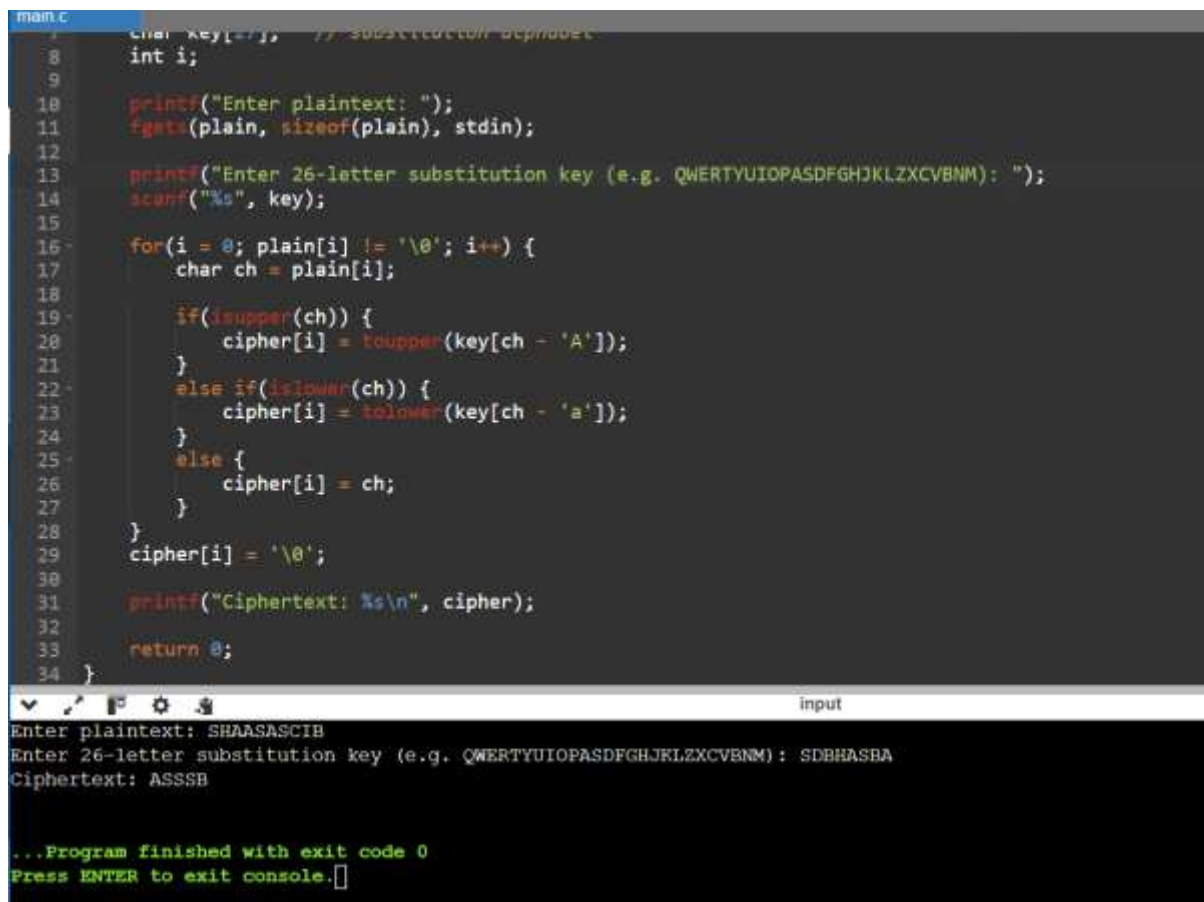
        if(isupper(ch)) {
            cipher[i] = toupper(key[ch - 'A']);
        }
        else if(islower(ch)) {
            cipher[i] = tolower(key[ch - 'a']);
        }
        else {
            cipher[i] = ch;
        }
    }
    cipher[i] = '\0';
```

```
printf("Ciphertext: %s\n", cipher);
```

```
return 0;
```

```
}
```

OUTPUT:



```
main.c
1  char key[26], // substitution alphabet
2  int i;
3
4
5
6
7
8
9
10 printf("Enter plaintext: ");
11 fgets(plain, sizeof(plain), stdin);
12
13 printf("Enter 26-letter substitution key (e.g. QWERTYUIOPASDFGHJKLZXCVBNM): ");
14 scanf("%s", key);
15
16 for(i = 0; plain[i] != '\0'; i++) {
17     char ch = plain[i];
18
19     if(isupper(ch)) {
20         cipher[i] = toupper(key[ch - 'A']);
21     }
22     else if(islower(ch)) {
23         cipher[i] = tolower(key[ch - 'a']);
24     }
25     else {
26         cipher[i] = ch;
27     }
28 }
29 cipher[i] = '\0';
30
31 printf("Ciphertext: %s\n", cipher);
32
33 return 0;
34 }
```

input

```
Enter plaintext: SHAASASCIB
Enter 26-letter substitution key (e.g. QWERTYUIOPASDFGHJKLZXCVBNM): SDBHASBA
Ciphertext: ASSSB

...Program finished with exit code 0
Press ENTER to exit console.
```

EXPERIMENT 3:

CODE:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

char matrix[5][5];

void generateMatrix(char key[]) {
    int used[26] = {0};
    int i, j, k = 0;
    char temp[25];

    for(i = 0; key[i] != '\0'; i++) {
        char ch = toupper(key[i]);
        if(ch == 'J') ch = 'I';
        if(ch >= 'A' && ch <= 'Z' && !used[ch - 'A']) {
            temp[k++] = ch;
            used[ch - 'A'] = 1;
        }
    }

    for(i = 0; i < 26; i++) {
        if(i == ('J' - 'A')) continue;
        if(!used[i]) temp[k++] = i + 'A';
    }

    k = 0;
    for(i = 0; i < 5; i++) {
        for(j = 0; j < 5; j++) {
```

```

        matrix[i][j] = temp[k++];
    }
}
}

```

```

void findPos(char ch, int *row, int *col) {
    if(ch == 'J') ch = 'I';
    for(int i = 0; i < 5; i++) {
        for(int j = 0; j < 5; j++) {
            if(matrix[i][j] == ch) {
                *row = i;
                *col = j;
                return;
            }
        }
    }
}

```

```

void prepareText(char plain[], char prepared[]) {
    int i, j = 0;
    for(i = 0; plain[i] != '\0'; i++) {
        if(isalpha(plain[i])) {
            prepared[j++] = toupper(plain[i]);
        }
    }
    prepared[j] = '\0';

    for(i = 0; i < j; i++) {
        if(prepared[i] == 'J') prepared[i] = 'I';
    }
}

```

```

char temp[100];
int k = 0;
for(i = 0; i < j; i++) {
    temp[k++] = prepared[i];
    if(prepared[i] == prepared[i+1]) {
        temp[k++] = 'X';
    }
}
temp[k] = '\0';

if(k % 2 != 0) temp[k++] = 'X';
temp[k] = '\0';

strcpy(prepared, temp);
}

void encryptPlayfair(char plain[], char cipher[]) {
    int i, r1, c1, r2, c2, k = 0;
    for(i = 0; plain[i] != '\0'; i += 2) {
        findPos(plain[i], &r1, &c1);
        findPos(plain[i+1], &r2, &c2);

        if(r1 == r2) {
            cipher[k++] = matrix[r1][(c1 + 1) % 5];
            cipher[k++] = matrix[r2][(c2 + 1) % 5];
        }
        else if(c1 == c2) {
            cipher[k++] = matrix[(r1 + 1) % 5][c1];
            cipher[k++] = matrix[(r2 + 1) % 5][c2];
        }
    }
}

```



```

    }
    else {
        cipher[k++] = matrix[r1][c2];
        cipher[k++] = matrix[r2][c1];
    }
}
cipher[k] = '\0';
}

int main() {
    char key[50], plain[100], prepared[100], cipher[100];

    printf("Enter key: ");
    scanf("%s", key);
    getchar();

    printf("Enter plaintext: ");
    fgets(plain, sizeof(plain), stdin);

    generateMatrix(key);
    prepareText(plain, prepared);
    encryptPlayfair(prepared, cipher);

    printf("Prepared text: %s\n", prepared);
    printf("Ciphertext: %s\n", cipher);

    return 0;
}

```

OUTPUT:

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 char matrix[5][5];
6
7 void generateMatrix(char key[]) {
8     int used[26] = {0};
9     int i, j, k = 0;
10    char temp[25];
11
12    for(i = 0; key[i] != '\0'; i++) {
13        char ch = toupper(key[i]);
14        if(ch == 'J') ch = 'I';
15        if(ch >= 'A' && ch <= 'Z' && !used[ch - 'A']) {
16            temp[k++] = ch;
17            used[ch - 'A'] = 1;
18        }
19    }
20
21    for(i = 0; i < 26; i++) {
22        if(i == ('J' - 'A')) continue;
23        if(!used[i]) temp[k++] = i + 'A';
24    }
25
26    k = 0;
27    for(i = 0; i < 5; i++) {
28        for(j = 0; j < 5; j++) {
29            matrix[i][j] = temp[k++];
30        }
31    }
32}
33
34 int main() {
35    char key[100], plain[100], cipher[100];
36    int i, j = 0;
37
38    printf("Enter key: ");
39    fgets(key, sizeof(key), stdin);
40
41    printf("Enter plaintext: ");
42    fgets(plain, sizeof(plain), stdin);
43
44    generateMatrix(key);
45
46    printf("Cipher text: ");
47    for(i = 0; i < strlen(plain); i++) {
48        cipher[j++] = plain[i];
49        cipher[j++] = matrix[i/5][i%5];
50    }
51    cipher[j] = '\0';
52
53    printf("Cipher text: %s\n", cipher);
54
55    return 0;
56}
```

Enter key: 6
Enter plaintext: ABSFIAB
Cipher text: ABSFIABX
Cipher text: CFQHFDCW

Program finished with exit code 0

EXPERIMENT 4:

CODE:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
int main() {
```

```
    char plain[100], key[100], cipher[100];
```

```
    int i, j = 0;
```

```
    printf("Enter plaintext: ");
```

```
    fgets(plain, sizeof(plain), stdin);
```

```
    printf("Enter key: ");
```

```

scanf("%s", key);

int keyLen = strlen(key);

for(i = 0; plain[i] != '\0'; i++) {
    char ch = plain[i];

    if(isalpha(ch)) {
        int shift = toupper(key[j % keyLen]) - 'A';

        if(isupper(ch))
            cipher[i] = ((ch - 'A' + shift) % 26) + 'A';
        else
            cipher[i] = ((ch - 'a' + shift) % 26) + 'a';

        j++;
    } else {
        cipher[i] = ch;
    }
}

cipher[i] = '\0';

printf("Ciphertext: %s\n", cipher);

return 0;
}

```

OUTPUT:

```
12 printf("Enter key. "),
13 scanf("%s", key);
14
15 int keyLen = strlen(key);
16
17 for(i = 0; plain[i] != '\0'; i++) {
18     char ch = plain[i];
19
20     if(isalpha(ch)) {
21         int shift = toupper(key[j % keyLen]) - 'A';
22
23         if(isupper(ch))
24             cipher[i] = ((ch - 'A' + shift) % 26) + 'A';
25         else
26             cipher[i] = ((ch - 'a' + shift) % 26) + 'a';
27
28         j++;
29     } else {
30         cipher[i] = ch;
31     }
32 }
33
34 cipher[i] = '\0';
35
36 printf("Ciphertext: %s\n", cipher);
37
38 return 0;
39 }
```

Enter plaintext: FOFNEO
Enter key: DSA
Ciphertext: IGFQWO

EXPERIMENT 5:

CODE:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
int gcd(int a, int b) {
    while(b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

```
int main() {  
    char plain[100], cipher[100];  
    int a, b, i;  
  
    printf("Enter plaintext: ");  
    fgets(plain, sizeof(plain), stdin);  
  
    printf("Enter value of a: ");  
    scanf("%d", &a);  
  
    printf("Enter value of b: ");  
    scanf("%d", &b);  
  
    if(gcd(a, 26) != 1) {  
        printf("Invalid value of a! It must be coprime with 26.\n");  
        return 1;  
    }  
  
    for(i = 0; plain[i] != '\0'; i++) {  
        char ch = plain[i];  
  
        if(isupper(ch)) {  
            int p = ch - 'A';  
            cipher[i] = ((a * p + b) % 26) + 'A';  
        }  
        else if(islower(ch)) {  
            int p = ch - 'a';  
            cipher[i] = ((a * p + b) % 26) + 'a';  
        }  
    }  
}
```

```

        else {
            cipher[i] = ch;
        }
    }

    cipher[i] = '\0';

    printf("Ciphertext: %s\n", cipher);

    return 0;
}

```

OUTPUT:

```

26
27 if(gcd(a, 26) != 1) {
28     printf("Invalid value of a! It must be coprime with 26.\n");
29     return 1;
30 }
31
32 for(i = 0; plain[i] != '\0'; i++) {
33     char ch = plain[i];
34
35     if(isupper(ch)) {
36         int p = ch - 'A';
37         cipher[i] = ((a * p + b) % 26) + 'A';
38     }
39     else if(islower(ch)) {
40         int p = ch - 'a';
41         cipher[i] = ((a * p + b) % 26) + 'a';
42     }
43     else {
44         cipher[i] = ch;
45     }
46 }
47
48 cipher[i] = '\0';
49
50 printf("Ciphertext: %s\n", cipher);
51
52 return 0;
53 }

```

input

```

Enter plaintext: JIDFNABU
Enter value of a: DFA
Enter value of b: Invalid value of a! It must be coprime with 26.

..Program finished with exit code 1
Press ENTER to exit console.

```

EXPERIMENT 6

CODE:

```
#include<stdio.h>

#include<string.h>


int main()
{
    char cipher[200];

    printf("Enter Ciphertext: ");
    gets(cipher);

    printf("\nPossible Plaintext:\n");

    for(int i=0; cipher[i]; i++)
    {
        char ch=cipher[i];

        if(ch>='A'&&ch<='Z')
        {
            int c=ch-'A';

            int p=(9*(c-15+26))%26;

            printf("%c",p+'A');
        }
        else
            printf("%c",ch);
    }
```

```

return 0;
}

```

OUTPUT:

```

1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6     char cipher[256];
7
8     printf("Enter Ciphertext: ");
9     gets(cipher);
10
11     printf("\nPossible Plaintext:\n");
12
13     for(int i=0; cipher[i]; i++)
14     {
15         char ch=cipher[i];
16
17         if(ch>='A' && ch<='Z')
18         {
19             int c=ch-'A';
20
21             int p=(c-13-26)%26;
22
23             printf("%c",p+'A');
24         }
25         else
26             printf("%c",ch);
27     }
28 }

```

main.c:1: text+0x2a): warning: the 'gets' function is dangerous and should not be used.

Enter Ciphertext:

Possible Plaintext:

..Program finished with exit code 0

Press ENTER to exit console.

EXPERIMENT 7:

CODE:

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main()
```

```
{
```

```
    char cipher[1000];
```

```
    int freq[256]={0};
```

```
    printf("Enter Ciphertext:\n");
```

```
    gets(cipher);
```



```

for(int i=0; cipher[i]; i++)
    freq[(unsigned char)cipher[i]]++;

printf("\nCharacter Frequencies\n");

for(int i=0;i<256;i++)
{
    if(freq[i]!=0)
        printf("%c : %d\n",i,freq[i]);
}

return 0;
}

```

OUTPUT:

The screenshot shows the OnlineGDB interface. On the left is a sidebar with navigation links like 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main area displays the C++ code from the previous block. The code includes headers for `stdio.h` and `string.h`, defines a character array `cipher` and an integer array `freq`, and contains a `main` function that reads input, counts character frequencies, and prints them. The output field at the bottom right is labeled 'Input' and is currently empty.

EXPERIMENT 8:

CODE:

```
#include<stdio.h>
```

```
#include<string.h>
```

```
char cipher[]="CIPHERABDFGJKLMNOQSTUVWXYZ";
```

```
int main()
{
    char plain[100];

    printf("Enter Plaintext: ");

    gets(plain);

    printf("Ciphertext:\n");

    for(int i=0; plain[i]; i++)
    {
        if(plain[i]>='a' && plain[i]<='z')
            printf("%c", cipher[plain[i]-'a']);

        else if(plain[i]>='A' && plain[i]<='Z')
            printf("%c", cipher[plain[i]-'A']);

        else
            printf("%c", plain[i]);
    }

    return 0;
}
```

OUTPUT:

```
1 #include<stdio.h>
2 #include<string.h>
3
4 char cipher[]="CIPHERABDFGJKLHMQOSTVWXYZ";
5
6 int main()
7 {
8     char plain[100];
9
10    printf("Enter Plaintext: ");
11    gets(plain);
12    printf("Ciphertext:");
13
14    for(int i=0; plain[i]; i++)
15    {
16        if(plain[i]!='a' && plain[i]!='z')
17            printf("%c", cipher[plain[i]-'a']);
18        else if(plain[i]!='A' && plain[i]!='Z')
19            printf("%c", cipher[plain[i]-'A']);
20        else
21            printf("%c", plain[i]);
22    }
23
24    printf("\n");
25
26 }
```

main.c(.text+0x24): warning: the 'gets' function is dangerous and should not be used.

Enter Plaintext: NOFALCA

Ciphertext: LJBEQJPC

...Program finished with exit code 0

Press ENTER to exit console.

EXPERIMENT 9:

CODE:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Playfair Cipher Decryption\n\n");
```

```
    printf("Use the Playfair matrix,\n");
```

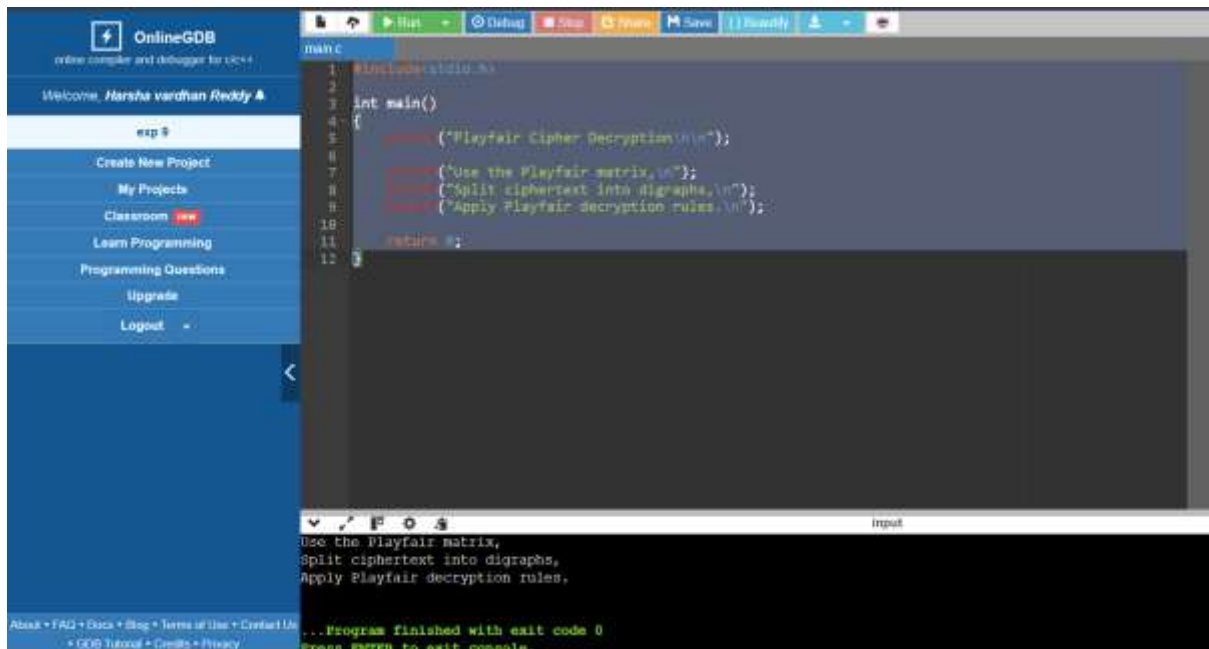
```
    printf("Split ciphertext into digraphs,\n");
```

```
    printf("Apply Playfair decryption rules.\n");
```

```
    return 0;
```

```
}
```

OUTPUT:



EXPERIMENT 10 :

CODE:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Playfair Encryption\n\n");
```

```
    printf("Matrix:\n");
```

```
    printf("M F H I K\n");
```

```
    printf("U N O P Q\n");
```

```
    printf("Z V W X Y\n");
```

```
    printf("E L A R G\n");
```

```
    printf("D S T B C\n\n");
```

```
    printf("Plaintext:\n");
```

```
    printf("Must see you over Cadogan West Coming at once\n\n");
```

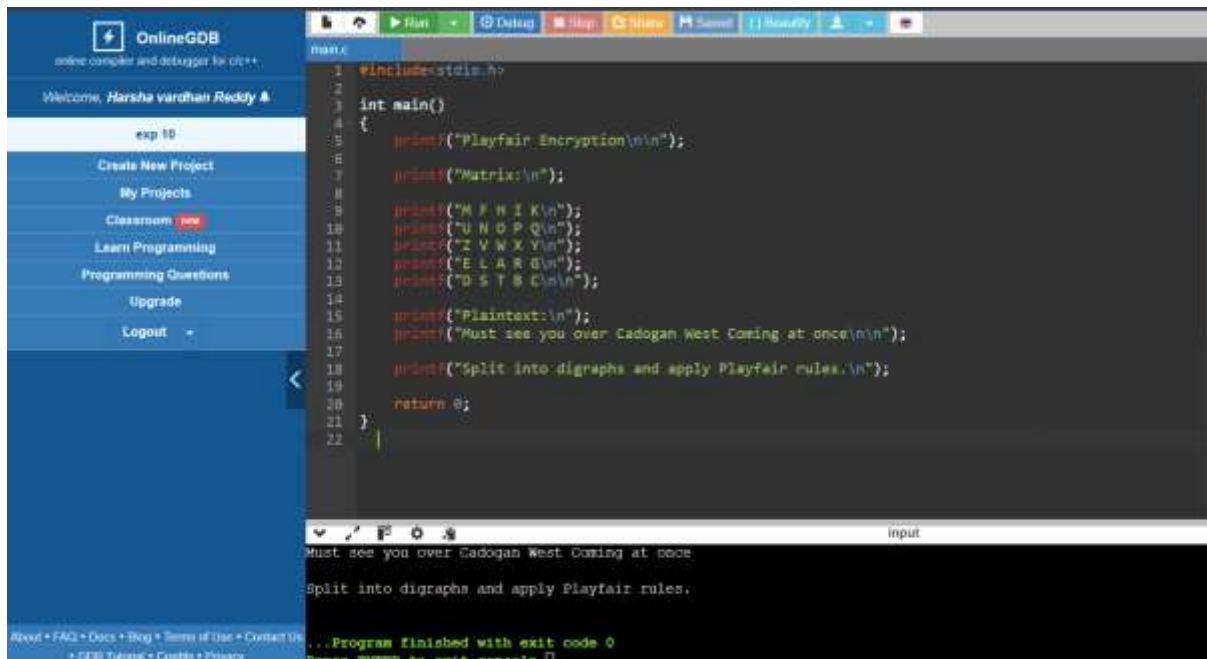
```
printf("Split into digraphs and apply Playfair rules.\n");
```

```
return 0;
```

```
}
```

```
-
```

OUTPUT:



```
main.c
1 #include<stdio.h>
2
3 int main()
4 {
5     printf("Playfair Encryption\n\n");
6     printf("Matrix:\n");
7
8     printf("M P N I K\n");
9     printf("U N O P Q\n");
10    printf("Z V W X Y\n");
11    printf("E L A R G\n");
12    printf("D S T B C\n");
13
14    printf("Plaintext:\n");
15    printf("Must see you over Cadogan West Coming at once\n\n");
16
17    printf("Split into digraphs and apply Playfair rules.\n");
18
19    return 0;
20 }
21
22
```

input

```
Must see you over Cadogan West Coming at once
Split into digraphs and apply Playfair rules.
```

...Program finished with exit code 0
Press ENTER to exit console.

EXPERIMENT 11:

CODE:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
printf("Playfair Cipher\n\n");
```

```
printf("Ignoring equivalent keys:\n");
```

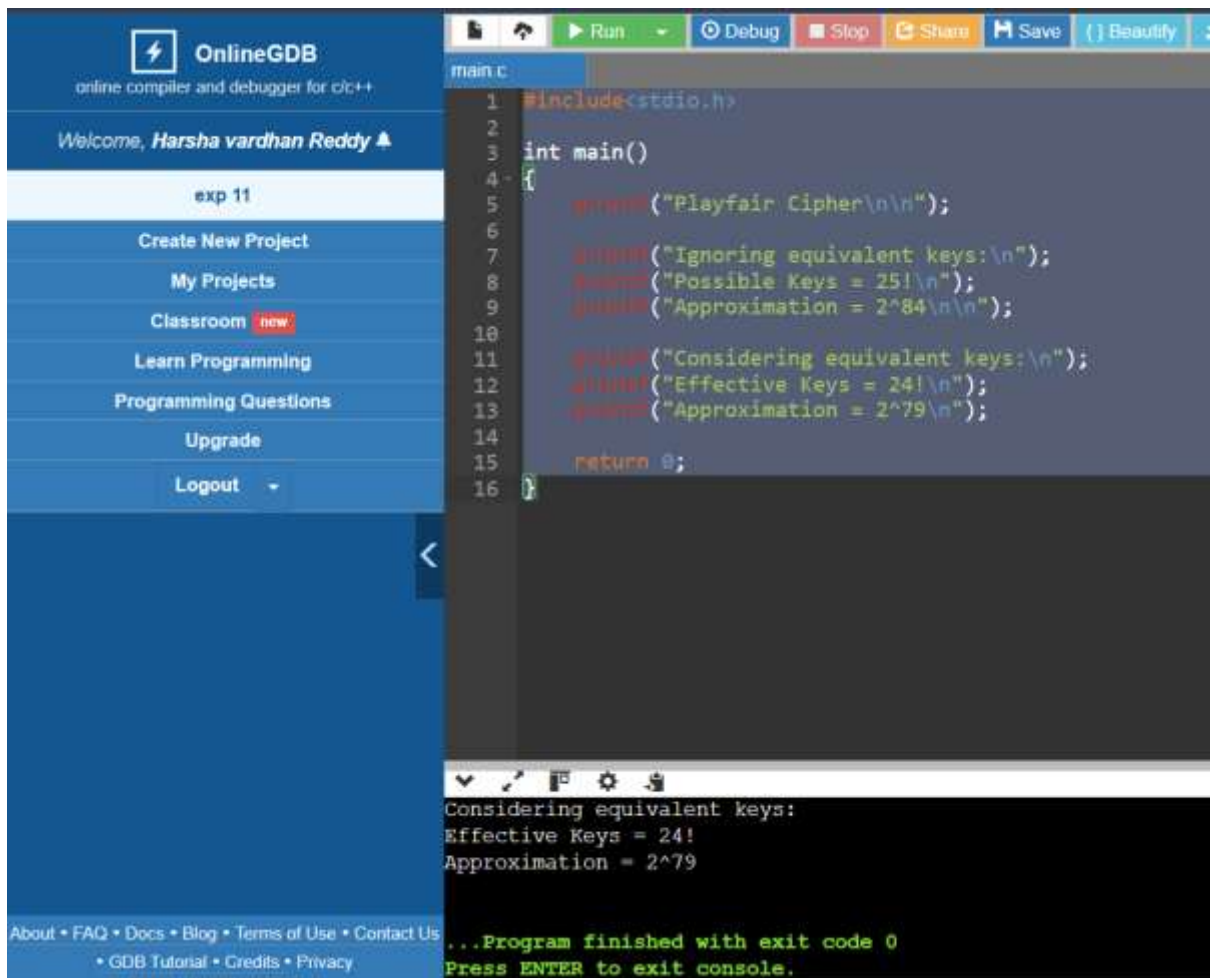
```
printf("Possible Keys = 25!\n");  
printf("Approximation = 2^84\n\n");
```

```
printf("Considering equivalent keys:\n");  
printf("Effective Keys = 24!\n");  
printf("Approximation = 2^79\n");
```

```
return 0;
```

```
}
```

OUTPUT:



The screenshot displays the OnlineGDB interface. On the left is a sidebar with the OnlineGDB logo, user name 'Harsha vardhan Reddy', and navigation links like 'exp 11', 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main area shows a C program with the following code:

```
1 #include<stdio.h>  
2  
3 int main()  
4 {  
5     printf("Playfair Cipher\n\n");  
6  
7     printf("Ignoring equivalent keys:\n");  
8     printf("Possible Keys = 25!\n");  
9     printf("Approximation = 2^84\n\n");  
10  
11    printf("Considering equivalent keys:\n");  
12    printf("Effective Keys = 24!\n");  
13    printf("Approximation = 2^79\n");  
14  
15    return 0;  
16 }
```

At the bottom, the console output shows the execution results:

```
Considering equivalent keys:  
Effective Keys = 24!  
Approximation = 2^79  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

EXPERIMENT 12:

CODE:

```
#include<stdio.h>

#include<string.h>


int main()
{
    int key[2][2]={{9,4},{5,7}};

    char plain[100];

    printf("Enter plaintext (even length): ");

    scanf("%s",plain);

    printf("Ciphertext: ");

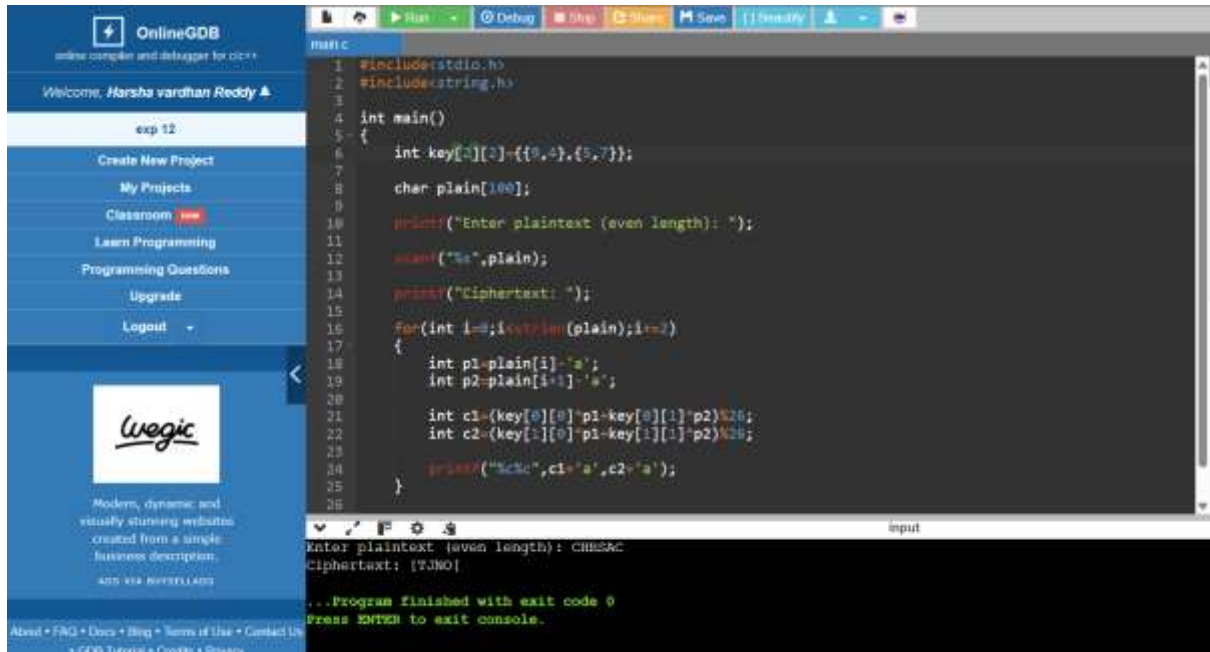
    for(int i=0;i<strlen(plain);i+=2)
    {
        int p1=plain[i]-'a';
        int p2=plain[i+1]-'a';

        int c1=(key[0][0]*p1+key[0][1]*p2)%26;
        int c2=(key[1][0]*p1+key[1][1]*p2)%26;

        printf("%c%c",c1+'a',c2+'a');
    }

    return 0;
}
```

OUTPUT:



```
1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6     int key[2][2]={0,0},{5,7};
7     char plain[100];
8
9     printf("Enter plaintext (even length): ");
10    scanf("%s",plain);
11    printf("Ciphertext: ");
12
13    for(int i=0;i<strlen(plain);i+=2)
14    {
15        int p1=plain[i]-'a';
16        int p2=plain[i+1]-'a';
17
18        int c1=(key[0][0]*p1+key[0][1]*p2)%26;
19        int c2=(key[1][0]*p1+key[1][1]*p2)%26;
20
21        printf("%c%c",c1+'a',c2+'a');
22    }
23
24    ...Program finished with exit code 0
25    Press ENTER to exit console.
```

EXPERIMENT 13:

CODE:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Known Plaintext Attack on Hill Cipher\n\n");
```

```
    printf("If attacker knows plaintext and ciphertext pairs,\n");
```

```
    printf("they form matrix equation:\n\n");
```

```
    printf("C = K * P (mod 26)\n\n");
```

```
    printf("Using matrix inversion,\n");
```

```
    printf("K = C * P^-1 (mod 26)\n\n");
```

```
    printf("Hence Hill cipher is vulnerable.\n");
```



```

return 0;
}

```

OUTPUT:

```

1 #include<stdio.h>
2
3 int main()
4 {
5     printf("Known Plaintext Attack on Hill Cipher\n");
6
7     printf("If attacker knows plaintext and ciphertext pairs,\n");
8     printf("they form matrix equation:\n");
9
10    printf("C = K * P (mod 26)\n");
11
12    printf("Using matrix inversion,\n");
13    printf("K = C * P^-1 (mod 26)\n");
14
15    printf("Hence Hill cipher is vulnerable.\n");
16
17    return 0;
18 }

```

R = C * P⁻¹ (mod 26)
 Hence Hill cipher is vulnerable.
 ...Program finished with exit code 0
 Press ENTER to exit console.

EXPERIMENT 14:

CODE:

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main()
```

```
{
```

```
    char plain[]="sendmoremoney";
```

```
    int key[]={9,0,1,7,23,15,21,14,11,11,2,8,9};
```

```
    printf("Ciphertext: ");
```

```
    for(int i=0;i<13;i++)
```

```

{
    int p=plain[i]-'a';

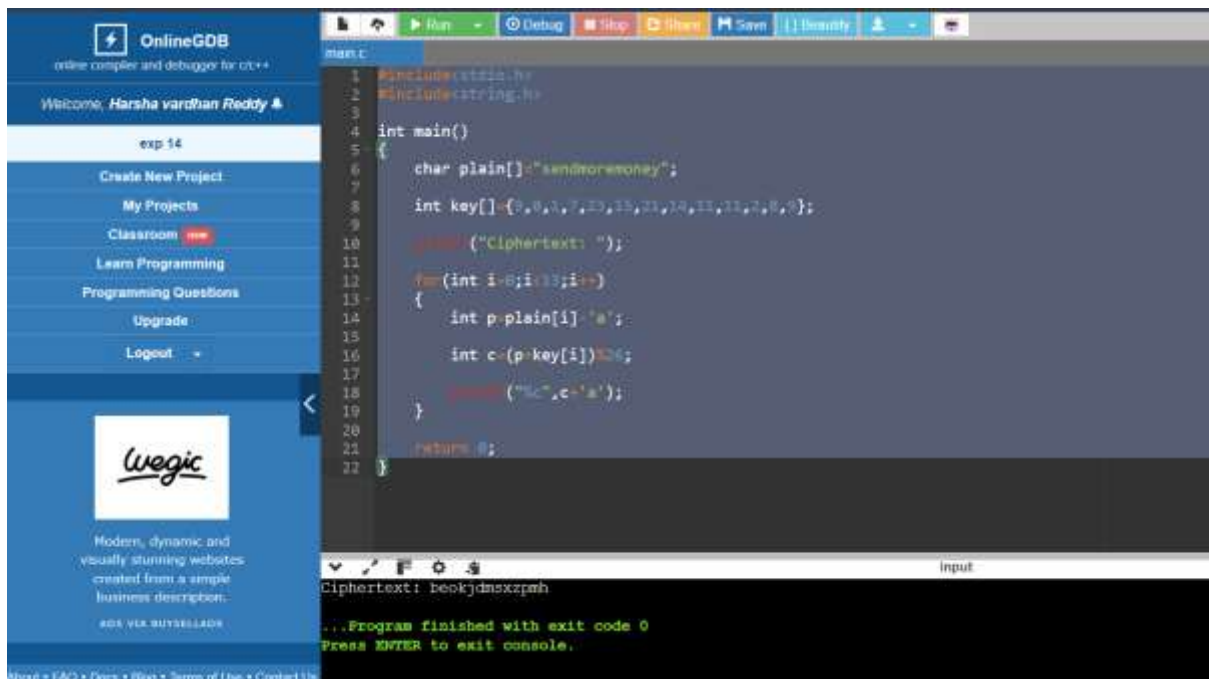
    int c=(p+key[i])%26;

    printf("%c",c+'a');
}

return 0;
}

```

OUTPUT:



```

OnlineGDB
online compiler and debugger for c/c++

Welcome, Harsha vardhan Reddy ▲

exp 14

Create New Project
My Projects
Classroom
Learn Programming
Programming Questions
Upgrade
Logout

w3gic
Modern, dynamic and
visually stunning websites
created from a simple
business description.
AOR VLA BUYSELLAOR

About • FAQ • Docs • Blog • Terms of Use • Contact Us

main.c
1 #include<stdio.h>
2 #include<string.h>
3
4 int main()
5 {
6     char plain[]="sandworexoney";
7
8     int key[]={0,0,1,7,23,15,21,10,11,11,7,0,0};
9
10    printf("Ciphertext: ");
11
12    for(int i=0;i<13;i++)
13    {
14        int p=plain[i]-'a';
15
16        int c=(p+key[i])%26;
17
18        printf("%c",c+'a');
19    }
20
21    return 0;
22 }

Ciphertext: beekjdnsxzmh
...Program finished with exit code 0
Press ENTER to exit console.

```

EXPERIMENT 15:

CODE:

```

#include<stdio.h>

#include<string.h>

#include<ctype.h>

int main()

```

```

{
    char cipher[500];

    int top;

    printf("Enter Ciphertext:\n");
    gets(cipher);

    printf("Enter number of possible plaintexts: ");
    scanf("%d",&top);

    if(top>26)
        top=26;

    for(int key=0;key<top;key++)
    {
        printf("\nKey %d : ",key);

        for(int i=0;i<strlen(cipher);i++)
        {
            char ch=cipher[i];

            if(isalpha(ch))
            {
                ch=toupper(ch);

                ch=((ch-'A'-key+26)%26)+'A';
            }

            printf("%c",ch);
        }
    }
}

```

```

printf("\n");

}

return 0;

}

```

OUTPUT:

```

1 #include<stdio.h>
2 #include<string.h>
3 #include<ctype.h>
4
5 int main()
6 {
7     char cipher[500];
8     int top;
9
10    printf("Enter Ciphertext:\n");
11    gets(cipher);
12
13    printf("Enter number of possible plaintexts: ");
14    scanf("%d",&top);
15
16    if(top>26)
17        top=26;
18
19    for(int key=0;key<top;key++)
20    {
21        printf("\nKey %d = ",key);
22
23        for(int i=0;i<strlen(cipher);i++)
24        {
25            char ch=cipher[i];
26

```

Input

```

667 | extern char *gets (char * __s) __wur __attribute__ deprecated __;
      |
/nix/store/7h3qmwgkw6z2r8lq4j5mks4l6r5x2cq-binutils-2.44/bin/ld: /tmp/ccwXCMnp.o: in function 'main':
main.c:(.text+0x26): warning: the 'gets' function is dangerous and should not be used.
Enter Ciphertext:
DLJHDSO
Enter number of possible plaintexts: 1

```

EXPERIMENT 16:

CODE:

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

```

```
char englishFreq[] = "ETAOINSHRDLCLUMWFGYPBVKJXQZ";
```

```

int main()
{
    char cipher[1000];

```

```

int freq[26] = {0};
int index[26];
char map[26];

printf("Enter Ciphertext:\n");
fgets(cipher, sizeof(cipher), stdin);

int top;
printf("Enter number of possible plaintexts required: ");
scanf("%d",&top);

for(int i=0; cipher[i]; i++)
{
    if(isalpha(cipher[i]))
        freq[toupper(cipher[i])-'A']++;
}

for(int i=0;i<26;i++)
    index[i]=i;

for(int i=0;i<25;i++)
{
    for(int j=i+1;j<26;j++)
    {
        if(freq[index[j]]>freq[index[i]])
        {
            int t=index[i];
            index[i]=index[j];
            index[j]=t;
        }
    }
}

```

```
}  
}
```

```
for(int i=0;i<26;i++)  
    map[index[i]]=englishFreq[i];
```

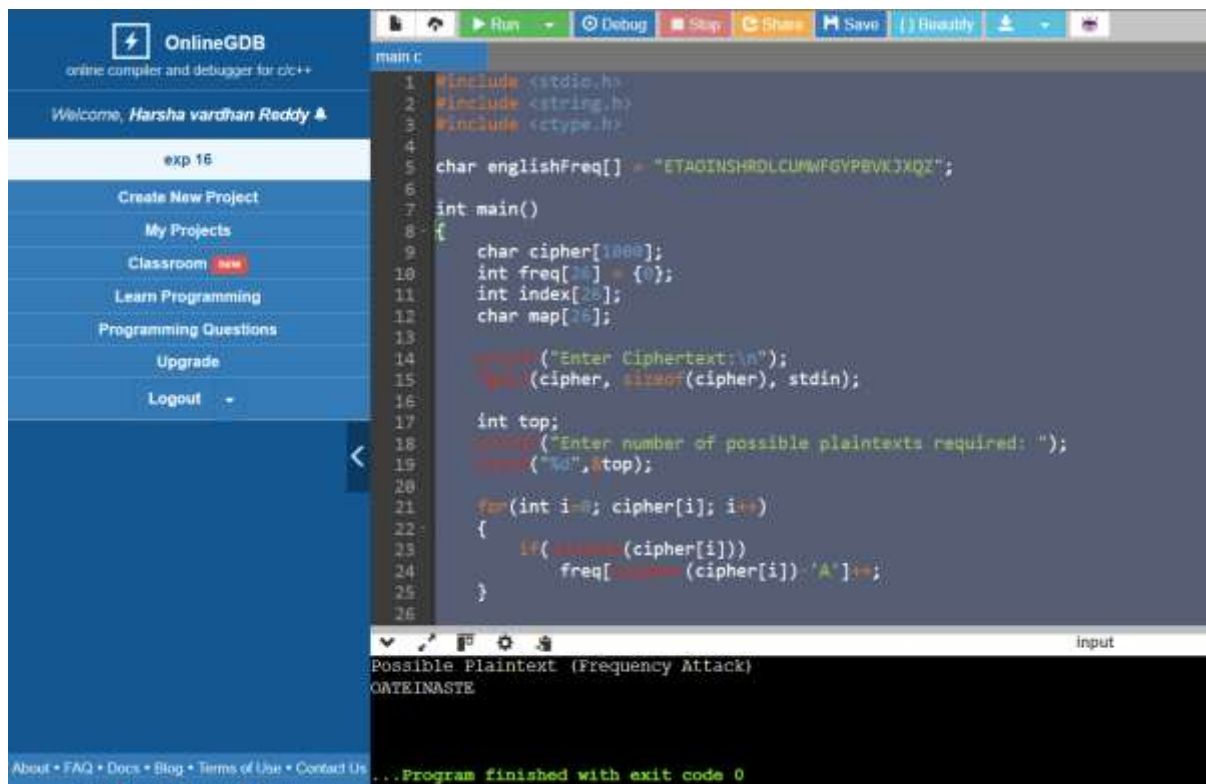
```
printf("\nPossible Plaintext (Frequency Attack)\n");
```

```
for(int i=0; cipher[i]; i++)  
{  
    if(isalpha(cipher[i]))  
        printf("%c",map[toupper(cipher[i])-'A']);  
    else  
        printf("%c",cipher[i]);  
}
```

```
printf("\n");
```

```
return 0;  
}
```

OUTPUT:



The screenshot shows the OnlineGDB interface. On the left is a sidebar with navigation links: 'exp 16', 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main area displays a C program for a frequency attack. The code includes headers for stdio, string, and ctype. It defines an array 'englishFreq' with the string 'ETAOINSHROLCUMWFGVPIBVKJXQZ'. The main function prompts the user to enter a ciphertext and the number of possible plaintexts. It then iterates through the ciphertext, counting the frequency of each letter. The output shows the ciphertext 'OATEINASTE' and the message '...Program finished with exit code 0'.

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 char englishFreq[] = "ETAOINSHROLCUMWFGVPIBVKJXQZ";
6
7 int main()
8 {
9     char cipher[1000];
10    int freq[26] = {0};
11    int index[26];
12    char map[26];
13
14    printf("Enter Ciphertext:\n");
15    scanf(cipher, sizeof(cipher), stdin);
16
17    int top;
18    printf("Enter number of possible plaintexts required: ");
19    scanf("%d", &top);
20
21    for(int i=0; cipher[i]; i++)
22    {
23        if(ctype_isalpha(cipher[i]))
24            freq[cipher[i] - 'A']++;
25    }
26
27    printf("Possible Plaintext (Frequency Attack)\n");
28    printf("OATEINASTE\n");
29
30    ...Program finished with exit code 0
```

EXPERIMENT 17:

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int keys[16];
```

```
    printf("Enter 16 subkeys:\n");
```

```
    for(int i=0;i<16;i++)
```

```
        scanf("%d",&keys[i]);
```

```
    printf("\nOrder of Keys for Decryption\n");
```

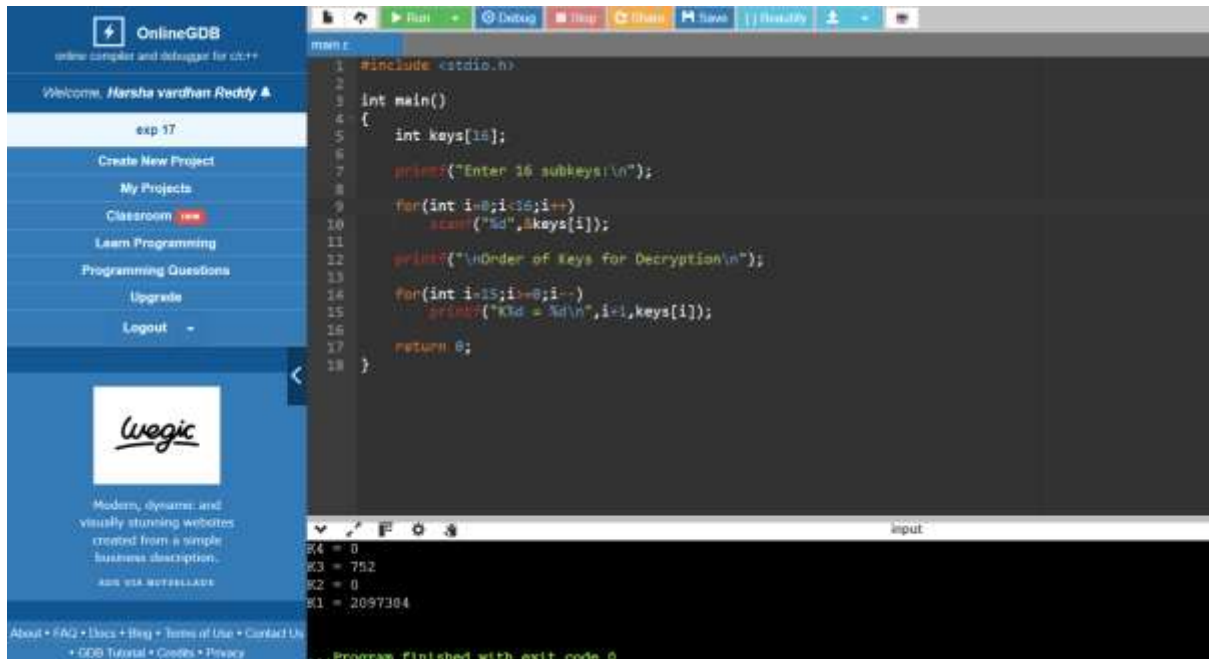
```
    for(int i=15;i>=0;i--)
```

```
printf("K%d = %d\n",i+1,keys[i]);
```

```
return 0;
```

```
}
```

OUTPUT:



The screenshot shows the OnlineGDB interface. On the left is a sidebar with navigation links like 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main area displays a C++ program for DES key generation. The code includes `<stdio.h>`, defines a `main` function, declares an array `keys[16]`, prompts the user to enter 16 subkeys, reads them into the array, prints the order of keys for decryption, and then prints the generated keys `K1` through `K4`. The output at the bottom shows the user's input: `K4 = 0`, `K3 = 752`, `K2 = 0`, and `K1 = 2097384`. The program ends with the message: `...Program finished with exit code 0`.

EXPERIMENT 18:

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
printf("DES Key Generation\n\n");
```

```
printf("56-bit key\n");
```

```
printf(" |\n");
```

```
printf("Split into C0 and D0 (28 bits each)\n");
```

```
printf(" |\n");
```



```

printf("Left Circular Shift\n");

printf(" \n");

printf("Apply PC-2\n");

printf(" \n");

printf("48-bit Round Key\n\n");


printf("First 24 bits <- C half\n");

printf("Second 24 bits <- D half\n");


return 0;
}

```

OUTPUT:

The screenshot displays the OnlineGDB environment. On the left is a sidebar with navigation links. The main area shows a C program for DES key generation. The code is as follows:

```

1 #include <stdio.h>
2
3 int main()
4 {
5     printf("DES Key Generation\n\n");
6
7     printf("56-bit key\n");
8     printf(" \n");
9     printf("Split into C and D (28 bits each)\n");
10    printf(" \n");
11    printf("Left Circular Shift\n");
12    printf(" \n");
13    printf("Apply PC-2\n");
14    printf(" \n");
15    printf("48-bit Round Key\n\n");
16
17    printf("First 24 bits <- C half\n");
18    printf("Second 24 bits <- D half\n");
19
20    return 0;
21 }

```

The output window at the bottom shows the first two lines of the program's execution:

```

First 24 bits <- C half
Second 24 bits <- D half

```

At the bottom of the output window, it states: "...Program finished with exit code 0. Press ENTER to exit console."

EXPERIMENT 19:

CODE:

```
#include <stdio.h>

unsigned char encrypt(unsigned char p,unsigned char key)
{
    return p^key;
}

int main()
{
    unsigned char key=0x5A;
    unsigned char iv=0xAA;

    unsigned char plaintext[4]={10,20,30,40};
    unsigned char cipher[4];

    unsigned char prev=iv;

    for(int i=0;i<4;i++)
    {
        cipher[i]=encrypt(plaintext[i]^prev,key);
        prev=cipher[i];
    }

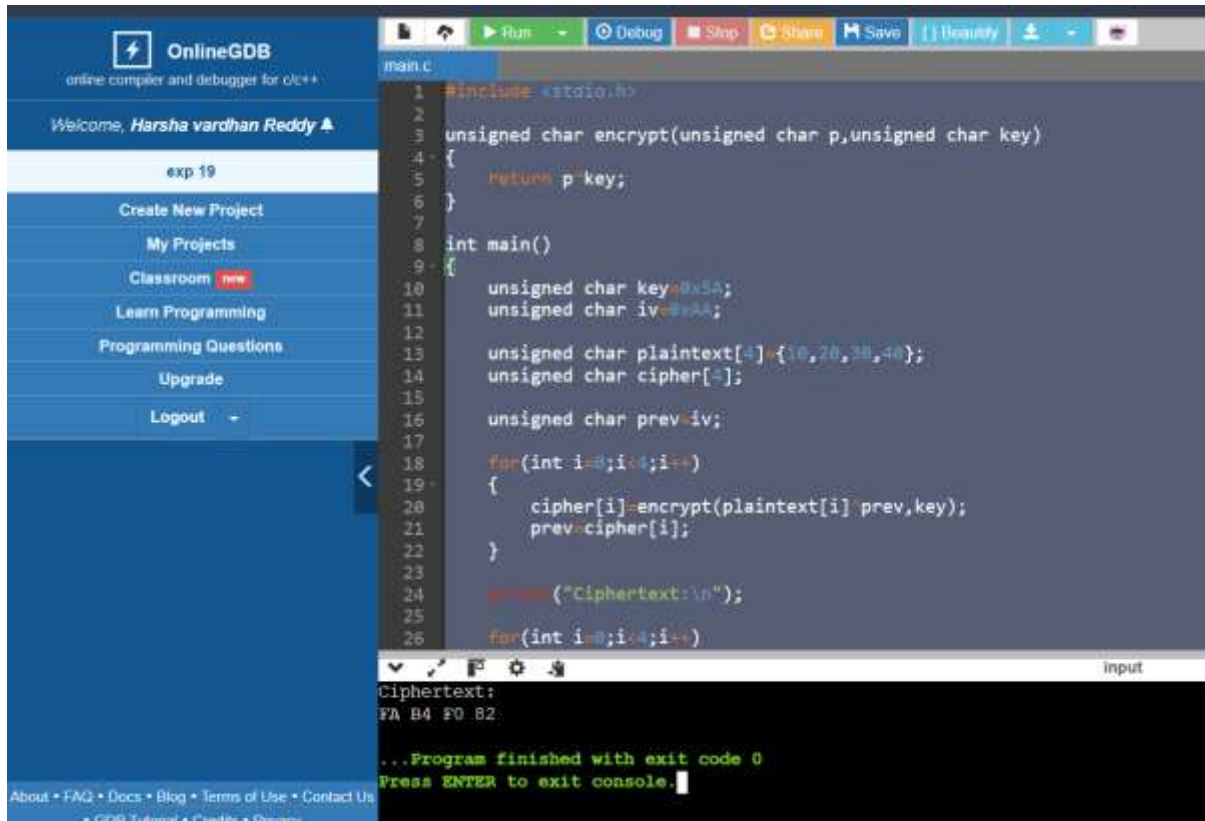
    printf("Ciphertext:\n");

    for(int i=0;i<4;i++)
        printf("%02X ",cipher[i]);

    return 0;
```

}

OUTPUT:



```
main.c
1 #include <stdio.h>
2
3 unsigned char encrypt(unsigned char p,unsigned char key)
4 {
5     return p ^ key;
6 }
7
8 int main()
9 {
10     unsigned char key=0x5A;
11     unsigned char iv=0xAA;
12
13     unsigned char plaintext[]={16,20,30,40};
14     unsigned char cipher[];
15
16     unsigned char prev=iv;
17
18     for(int i=0;i<4;i++)
19     {
20         cipher[i]=encrypt(plaintext[i]^prev,key);
21         prev=cipher[i];
22     }
23
24     printf("Ciphertext:\n");
25
26     for(int i=0;i<4;i++)
```

Ciphertext:
FA B4 F0 82

...Program finished with exit code 0
Press ENTER to exit console.

EXPERIMENT 20:

CODE:

```
#include <stdio.h>
```

```
#define SIZE 4
```

```
// Toy encryption/decryption using XOR
```

```
unsigned char encrypt(unsigned char data, unsigned char key)
```

```
{
```

```
    return data ^ key;
```

```
}
```

```
unsigned char decrypt(unsigned char data, unsigned char key)
```

```
{
```

```

    return data ^ key;
}

int main()
{
    unsigned char key = 0x5A;
    unsigned char iv = 0xAA;

    unsigned char plaintext[SIZE] = {10, 20, 30, 40};

    unsigned char ecbCipher[SIZE], ecbPlain[SIZE];
    unsigned char cbcCipher[SIZE], cbcPlain[SIZE];

    int i;

    /* ----- ECB Encryption ----- */
    for(i = 0; i < SIZE; i++)
        ecbCipher[i] = encrypt(plaintext[i], key);

    /* Introduce an error in the first ciphertext block */
    ecbCipher[0] ^= 0x01;

    /* ECB Decryption */
    for(i = 0; i < SIZE; i++)
        ecbPlain[i] = decrypt(ecbCipher[i], key);

    printf("ECB Mode\n");
    printf("Recovered Plaintext:\n");
    for(i = 0; i < SIZE; i++)
        printf("%d ", ecbPlain[i]);

```

```

printf("\n\n");

/* ----- CBC Encryption ----- */
unsigned char prev = iv;

for(i = 0; i < SIZE; i++)
{
    cbcCipher[i] = encrypt(plaintext[i] ^ prev, key);
    prev = cbcCipher[i];
}

/* Introduce an error in first ciphertext block */
cbcCipher[0] ^= 0x01;

/* CBC Decryption */
prev = iv;

for(i = 0; i < SIZE; i++)
{
    cbcPlain[i] = decrypt(cbcCipher[i], key) ^ prev;
    prev = cbcCipher[i];
}

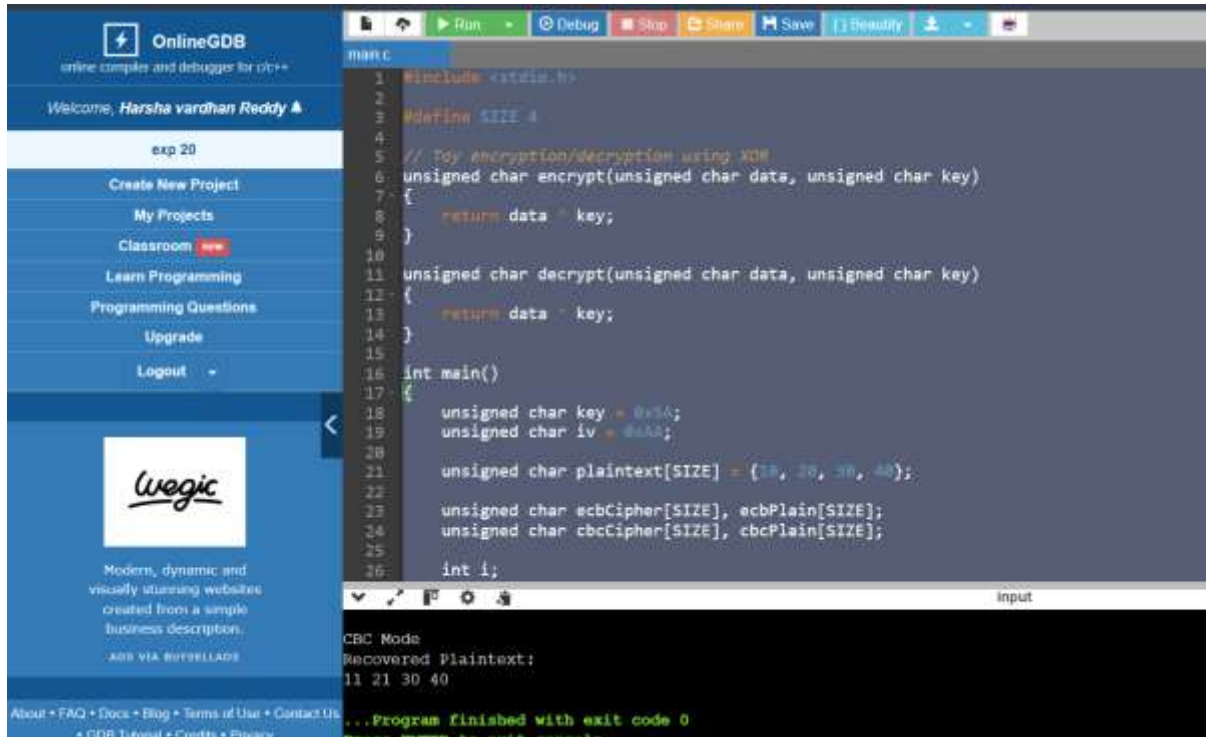
printf("CBC Mode\n");
printf("Recovered Plaintext:\n");
for(i = 0; i < SIZE; i++)
    printf("%d ", cbcPlain[i]);

return 0;

```

}

OUTPUT:



```
1 #include <stdio.h>
2
3 #define SIZE 4
4
5 // Toy encryption/decryption using XOR
6 unsigned char encrypt(unsigned char data, unsigned char key)
7 {
8     return data ^ key;
9 }
10
11 unsigned char decrypt(unsigned char data, unsigned char key)
12 {
13     return data ^ key;
14 }
15
16 int main()
17 {
18     unsigned char key = 0x5A;
19     unsigned char iv = 0xA5;
20
21     unsigned char plaintext[SIZE] = {11, 21, 30, 40};
22
23     unsigned char ecbCipher[SIZE], ecbPlain[SIZE];
24     unsigned char cbcCipher[SIZE], cbcPlain[SIZE];
25
26     int i;
27
28     // CBC Mode
29     printf("Recovered Plaintext:");
30     for (i = 0; i < SIZE; i++)
31     {
32         printf("%02X ", cbcPlain[i]);
33     }
34     printf("\n");
35
36     ...Program finished with exit code 0
37 Press ENTER to exit console.
```

EXPERIMENT 21:

CODE:

```
#include <stdio.h>
#include <string.h>
#define BLOCK_SIZE 8
void printBlock(unsigned char block[]) {
    for (int i = 0; i < BLOCK_SIZE; i++)
        printf("%02X ", block[i]);
    printf("\n");
}
void encryptBlock(unsigned char input[], unsigned char key[], unsigned char output[]) {
    for (int i = 0; i < BLOCK_SIZE; i++)
        output[i] = input[i] ^ key[i];
}
```

```

void decryptBlock(unsigned char input[], unsigned char key[], unsigned char output[]) {
    for (int i = 0; i < BLOCK_SIZE; i++)
        output[i] = input[i] ^ key[i];
}

```

```

void ECB(unsigned char plaintext[][BLOCK_SIZE], int blocks, unsigned char key[]) {
    unsigned char cipher[BLOCK_SIZE];
    printf("\nECB Encryption:\n");
    for (int i = 0; i < blocks; i++) {
        encryptBlock(plaintext[i], key, cipher);
        printf("Block %d: ", i + 1);
        printBlock(cipher);
    }
}

```

```

void CBC(unsigned char plaintext[][BLOCK_SIZE], int blocks, unsigned char key[],
unsigned char iv[]) {
    unsigned char temp[BLOCK_SIZE], cipher[BLOCK_SIZE];
    memcpy(temp, iv, BLOCK_SIZE);
    printf("\nCBC Encryption:\n");
    for (int i = 0; i < blocks; i++) {
        for (int j = 0; j < BLOCK_SIZE; j++)
            temp[j] ^= plaintext[i][j];
        encryptBlock(temp, key, cipher);
        printf("Block %d: ", i + 1);
        printBlock(cipher);
        memcpy(temp, cipher, BLOCK_SIZE);
    }
}

```

```

void CFB(unsigned char plaintext[][BLOCK_SIZE], int blocks, unsigned char key[],
unsigned char iv[]) {
    unsigned char feedback[BLOCK_SIZE];
    unsigned char cipher[BLOCK_SIZE];

```

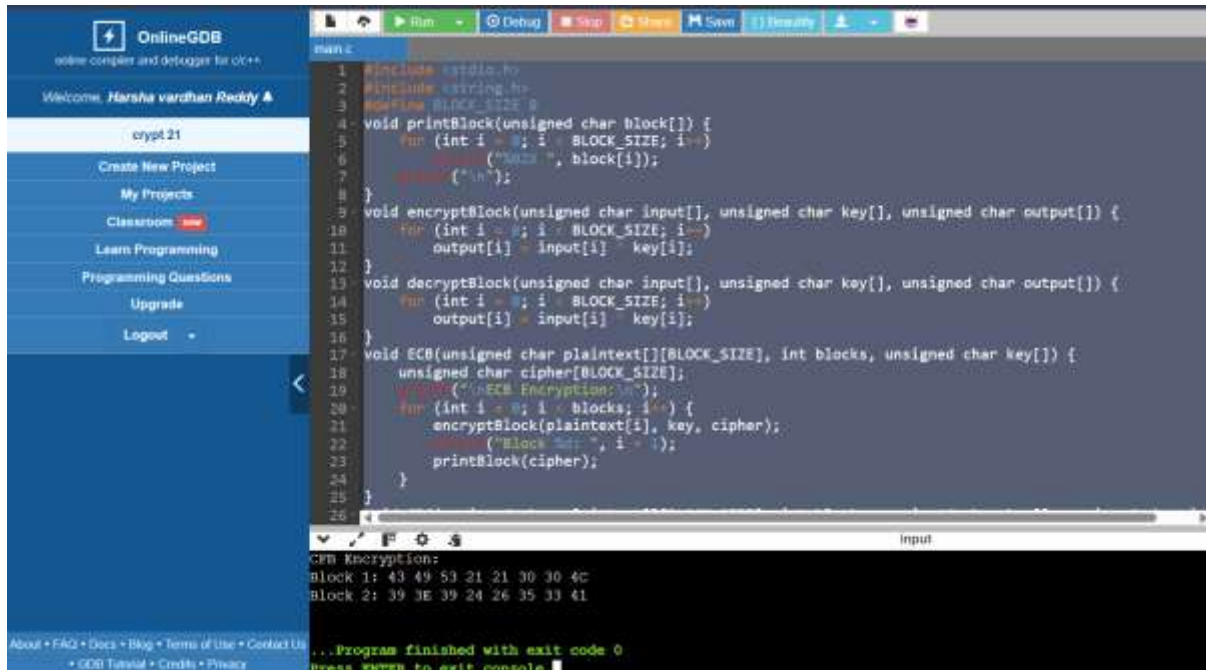
```

unsigned char temp[BLOCK_SIZE];
memcpy(feedback, iv, BLOCK_SIZE);
printf("\nCFB Encryption:\n");
for (int i = 0; i < blocks; i++) {
    encryptBlock(feedback, key, temp);
    for (int j = 0; j < BLOCK_SIZE; j++)
        cipher[j] = plaintext[i][j] ^ temp[j];
    printf("Block %d: ", i + 1);
    printBlock(cipher);
    memcpy(feedback, cipher, BLOCK_SIZE);
}
}

int main() {
    unsigned char key[BLOCK_SIZE] = {
        'K','E','Y','1','2','3','4','5'
    };
    unsigned char iv[BLOCK_SIZE] = {
        'T','N','T','T','V','E','C','I'
    };
    unsigned char plaintext[2][BLOCK_SIZE] = {
        {'A','B','C','D','E','F','G','H'},
        {'1','2','3','4','5','6','7','8'}
    };
    ECB(plaintext, 2, key);
    CBC(plaintext, 2, key, iv);
    CFB(plaintext, 2, key, iv);
    return 0;
}

```

OUTPUT:



```
1 #include <stdio.h>
2 #include <string.h>
3 #define BLOCK_SIZE 8
4 void printBlock(unsigned char block[]) {
5     for (int i = 0; i < BLOCK_SIZE; i++)
6         printf("%02X ", block[i]);
7     printf("\n");
8 }
9 void encryptBlock(unsigned char input[], unsigned char key[], unsigned char output[]) {
10     for (int i = 0; i < BLOCK_SIZE; i++)
11         output[i] = input[i] ^ key[i];
12 }
13 void decryptBlock(unsigned char input[], unsigned char key[], unsigned char output[]) {
14     for (int i = 0; i < BLOCK_SIZE; i++)
15         output[i] = input[i] ^ key[i];
16 }
17 void ECB(unsigned char plaintext[][BLOCK_SIZE], int blocks, unsigned char key[]) {
18     unsigned char cipher[BLOCK_SIZE];
19     printf("ECB Encryption:\n");
20     for (int i = 0; i < blocks; i++) {
21         encryptBlock(plaintext[i], key, cipher);
22         printf("Block %d: ", i + 1);
23         printBlock(cipher);
24     }
25 }
26
```

CFB Encryption:
Block 1: 43 49 53 21 21 30 30 4C
Block 2: 39 3E 39 24 26 35 33 41

...Program finished with exit code 0
Press ENTER to exit console.

EXPERIMENT 22:

CODE:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define BLOCK_SIZE 8
```

```
// Convert binary string to integer
```

```
int binToInt(char *bin) {
```

```
    int val = 0;
```

```
    while (*bin) {
```

```
        val = (val << 1) + (*bin - '0');
```

```
        bin++;
```

```
    }
```

```
    return val;
```

```
}
```

```
// Print an 8-bit binary number
```

```
void printBinary(unsigned char n) {  
    for (int i = 7; i >= 0; i--)  
        printf("%d", (n >> i) & 1);  
}
```

```
// -----
```

```
// Dummy S-DES encryption (for demonstration only)
```

```
// Replace this function with the complete S-DES algorithm if required
```

```
// -----
```

```
unsigned char SDES_Encrypt(unsigned char data, int key) {  
    return data ^ (key & 0xFF);  
}
```

```
unsigned char SDES_Decrypt(unsigned char data, int key) {  
    return data ^ (key & 0xFF);  
}
```

```
// CBC Encryption
```

```
void CBC_Encrypt(unsigned char plaintext[], unsigned char ciphertext[],  
                 int blocks, int key, unsigned char iv)  
{  
    unsigned char previous = iv;  
  
    for (int i = 0; i < blocks; i++) {  
        unsigned char x = plaintext[i] ^ previous;  
        ciphertext[i] = SDES_Encrypt(x, key);  
        previous = ciphertext[i];  
    }
```

```

    }
}

// CBC Decryption
void CBC_Decrypt(unsigned char ciphertext[], unsigned char plaintext[],
                 int blocks, int key, unsigned char iv)
{
    unsigned char previous = iv;

    for (int i = 0; i < blocks; i++) {
        unsigned char x = SDES_Decrypt(ciphertext[i], key);
        plaintext[i] = x ^ previous;
        previous = ciphertext[i];
    }
}

int main() {

    char keyStr[] = "0111111101";
    char ivStr[] = "10101010";

    int key = binToInt(keyStr);
    unsigned char iv = binToInt(ivStr);

    // Plaintext:
    // 00000001 00100011

    unsigned char plaintext[2];
    plaintext[0] = binToInt("00000001");
    plaintext[1] = binToInt("00100011");

```

```
unsigned char ciphertext[2];
unsigned char decrypted[2];

CBC_Encrypt(plaintext, ciphertext, 2, key, iv);

printf("Ciphertext:\n");
for (int i = 0; i < 2; i++) {
    printBinary(ciphertext[i]);
    printf(" ");
}

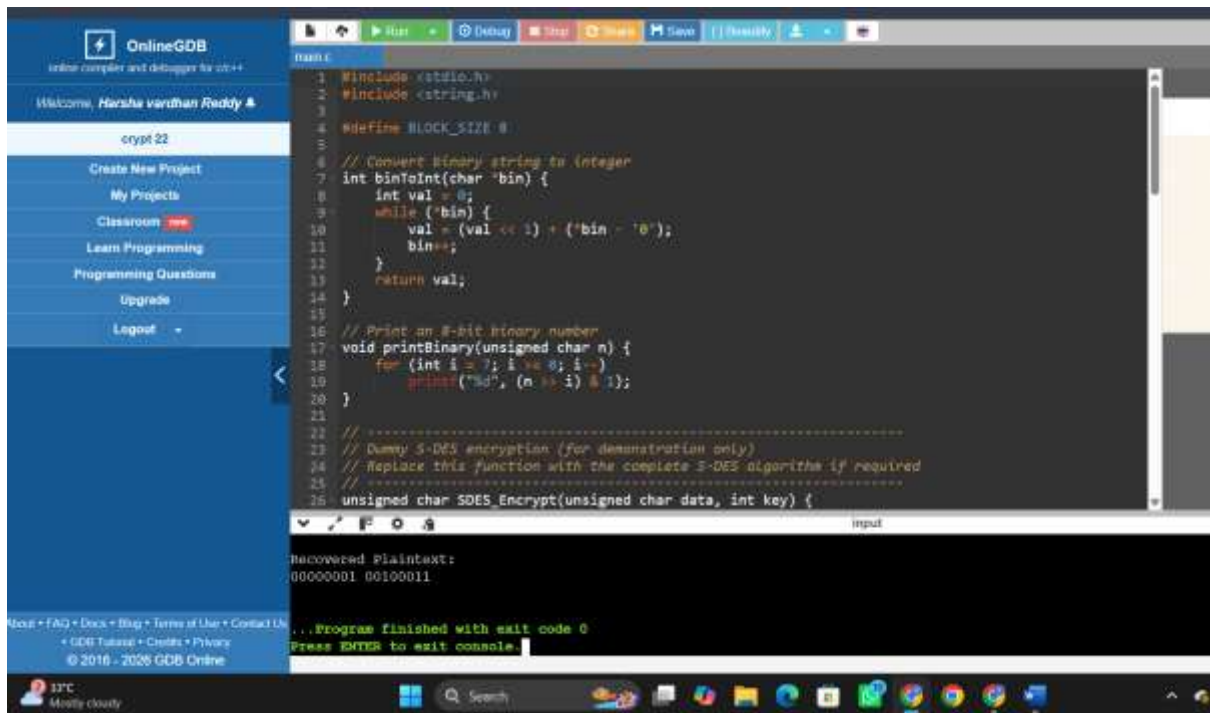
CBC_Decrypt(ciphertext, decrypted, 2, key, iv);

printf("\n\nRecovered Plaintext:\n");
for (int i = 0; i < 2; i++) {
    printBinary(decrypted[i]);
    printf(" ");
}

printf("\n");

return 0;
}
```

OUTPUT:



```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define BLOCK_SIZE 8
5
6 // Convert binary string to integer
7 int binToInt(char *bin) {
8     int val = 0;
9     while (*bin) {
10         val = (val << 1) + (*bin - '0');
11         bin++;
12     }
13     return val;
14 }
15
16 // Print an 8-bit binary number
17 void printBinary(unsigned char a) {
18     for (int i = 7; i >= 0; i--)
19         printf("%d", (a >> i) & 1);
20 }
21
22 // -----
23 // Dummy S-DES encryption (for demonstration only)
24 // Replace this function with the complete S-DES algorithm if required
25 // -----
26 unsigned char SDES_Encrypt(unsigned char data, int key) {
27     // ...
28 }
29
30 int main() {
31     unsigned char plaintext[BLOCK_SIZE] = {1, 2, 4};
32     unsigned char ciphertext[BLOCK_SIZE];
33     SDES_Encrypt(plaintext, key);
34     printBinary(ciphertext);
35     return 0;
36 }
```

Recovered Plaintext:
00000001 00100011

...Program finished with exit code 0
Press ENTER to exit console.

EXPERIMENT 23:

CODE:

```
#include <stdio.h>
```

```
#define BLOCKS 3
```

```
unsigned char affine_encrypt(unsigned char x)
```

```
{
    return (5 * x + 8) % 256;
}
```

```
int main()
```

```
{
    unsigned char plaintext[BLOCKS] = {1,2,4};
    unsigned char ciphertext[BLOCKS];
```

```
unsigned char decrypted[BLOCKS];
```

```
unsigned char counter = 0;
```

```
printf("Plaintext:\n");
```

```
for(int i=0;i<BLOCKS;i++)
```

```
    printf("%02X ",plaintext[i]);
```

```
printf("\n\nEncryption\n");
```

```
for(int i=0;i<BLOCKS;i++)
```

```
{
```

```
    unsigned char key = affine_encrypt(counter);
```

```
    ciphertext[i] = plaintext[i] ^ key;
```

```
    printf("Counter=%02X Key=%02X Cipher=%02X\n",
```

```
        counter,key,ciphertext[i]);
```

```
    counter++;
```

```
}
```

```
counter = 0;
```

```
printf("\nDecryption\n");
```

```
for(int i=0;i<BLOCKS;i++)
```

```
{
```

```
    unsigned char key = affine_encrypt(counter);
```

```
    decrypted[i] = ciphertext[i] ^ key;
```

```
    printf("Counter=%02X Plain=%02X\n",
```

```
        counter,decrypted[i]);
```

```
    counter++;
```

```
}
```

```
printf("\nRecovered Plaintext:\n");
```

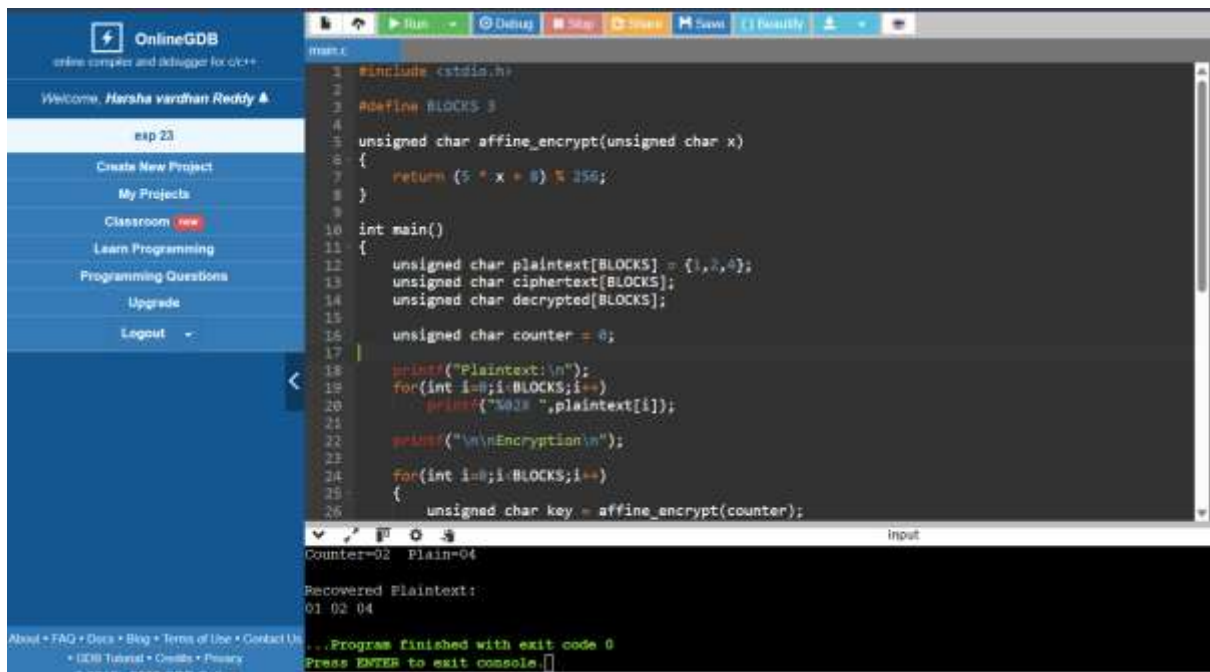
```
for(int i=0;i<BLOCKS;i++)
```

```
    printf("%02X ",decrypted[i]);
```

```
return 0;
```

```
}
```

OUTPUT:



```
main.c
1 #include <stdio.h>
2
3 #define BLOCKS 3
4
5 unsigned char affine_encrypt(unsigned char x)
6 {
7     return (x * 5 + 8) % 256;
8 }
9
10 int main()
11 {
12     unsigned char plaintext[BLOCKS] = {1,2,4};
13     unsigned char ciphertext[BLOCKS];
14     unsigned char decrypted[BLOCKS];
15
16     unsigned char counter = 0;
17
18     printf("Plaintext:\n");
19     for(int i=0;i<BLOCKS;i++)
20         printf("%02X ",plaintext[i]);
21
22     printf("\n\nEncryption\n");
23
24     for(int i=0;i<BLOCKS;i++)
25     {
26         unsigned char key = affine_encrypt(counter);
27
28         ciphertext[i] = affine_encrypt(plaintext[i] + key);
29         counter++;
30     }
31
32     printf("\n\nDecryption\n");
33
34     for(int i=0;i<BLOCKS;i++)
35     {
36         unsigned char key = affine_encrypt(counter);
37         decrypted[i] = affine_decrypt(ciphertext[i] + key);
38         counter++;
39     }
40
41     printf("\nRecovered Plaintext:\n");
42     for(int i=0;i<BLOCKS;i++)
43         printf("%02X ",decrypted[i]);
44
45     return 0;
46 }
```

Counter=02 Plain=04

Recovered Plaintext:
01 02 04

...Program finished with exit code 0
Press ENTER to exit console.

EXPERIMENT 24:

CODE:

```
#include <stdio.h>
```

```
int gcdExtended(int a, int b, int *x, int *y)
```

```
{
```

```
if(a==0)
{
    *x=0;
    *y=1;
    return b;
}
```

```
int x1,y1;
```

```
int gcd=gcdExtended(b%a,a,&x1,&y1);
```

```
*x=y1-(b/a)*x1;
```

```
*y=x1;
```

```
return gcd;
```

```
}
```

```
int main()
```

```
{
```

```
int e=31;
```

```
int phi=3480;
```

```
int x,y;
```

```
gcdExtended(e,phi,&x,&y);
```

```
int d=(x%phi+phi)%phi;
```

```
printf("Public Key\n");
```

```
printf("e=%d\n",e);
```



```
printf("n=3599\n\n");
```

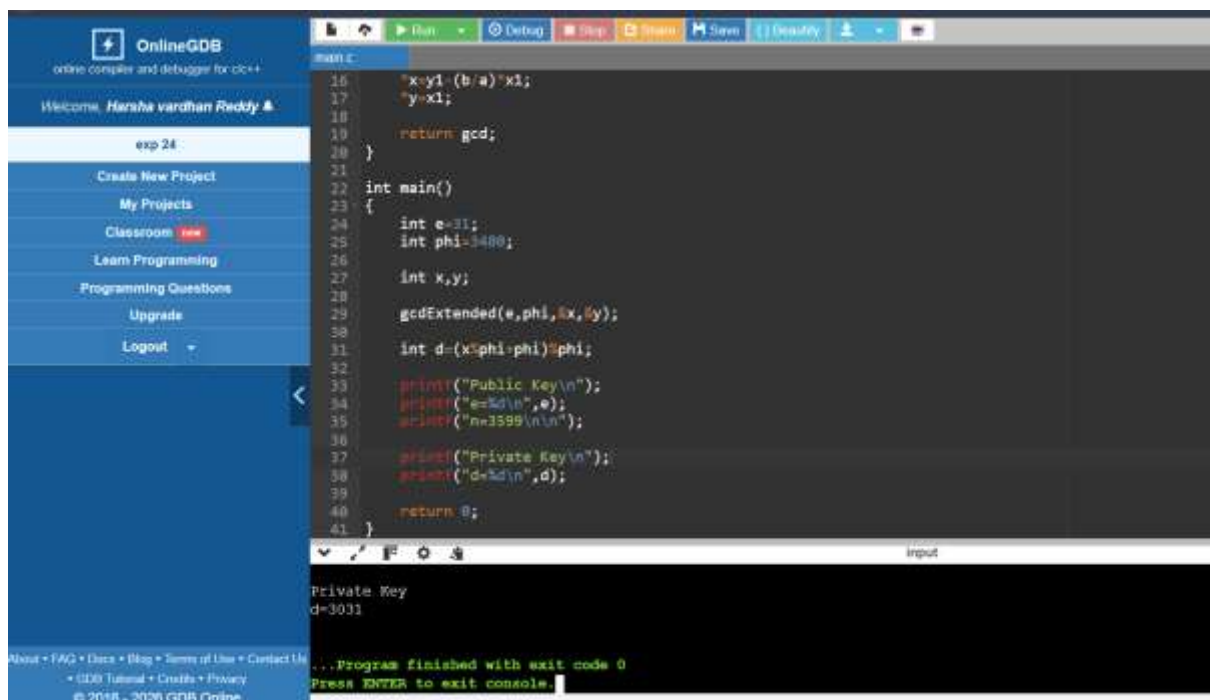
```
printf("Private Key\n");
```

```
printf("d=%d\n",d);
```

```
return 0;
```

```
}
```

OUTPUT:



The screenshot shows the OnlineGDB interface. The code in the editor is as follows:

```
16     *x=y1*(b/a)*x1;
17     *y=x1;
18
19     return gcd;
20 }
21
22 int main()
23 {
24     int e=31;
25     int phi=3480;
26
27     int x,y;
28
29     gcdExtended(e,phi,&x,&y);
30
31     int d=(x*phi-phi)%phi;
32
33     printf("Public Key\n");
34     printf("e=%d\n",e);
35     printf("n=3599\n\n");
36
37     printf("Private Key\n");
38     printf("d=%d\n",d);
39
40     return 0;
41 }
```

The output at the bottom of the console is:

```
Private Key
d=3011

...Program finished with exit code 0
Press ENTER to exit console.
```

EXPERIMENT 25:

CODE:

```
#include <stdio.h>
```

```
int gcd(int a,int b)
```

```
{
```

```
while(b!=0)
```

```
{
```

```
int t=b;
```

```
b=a%b;
```

```

        a=t;
    }
    return a;
}

int main()
{
    int n=3599;
    int plaintext=59; // Shares a factor with n

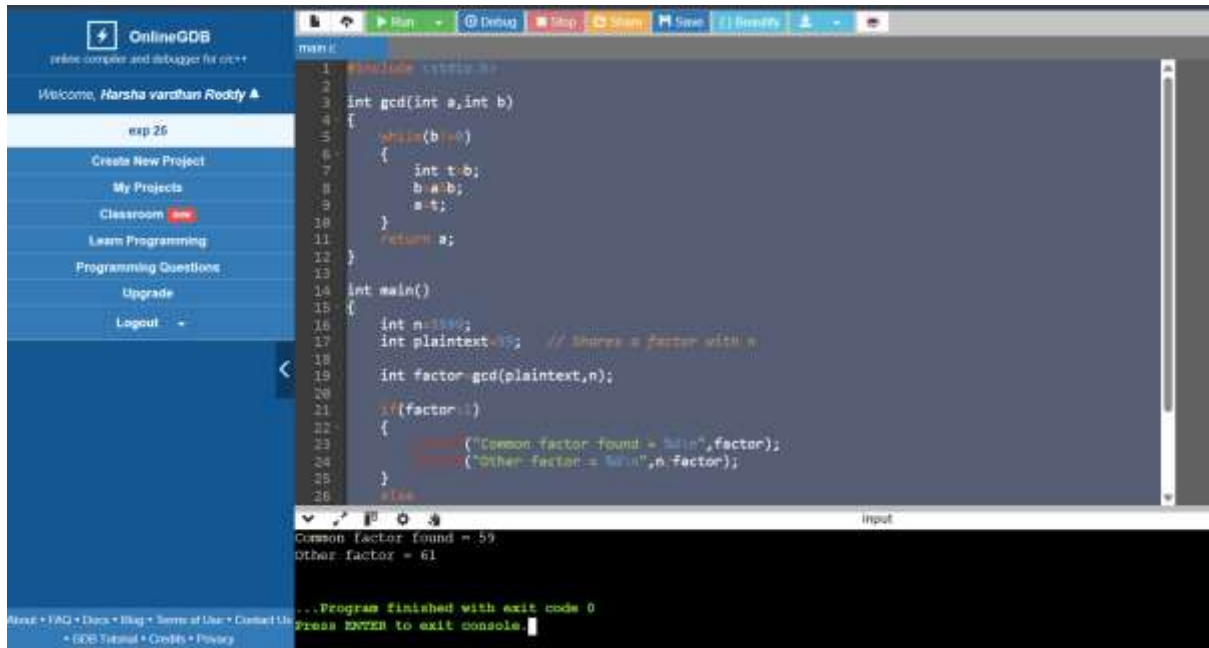
    int factor=gcd(plaintext,n);

    if(factor>1)
    {
        printf("Common factor found = %d\n",factor);
        printf("Other factor = %d\n",n/factor);
    }
    else
    {
        printf("No common factor found.\n");
    }

    return 0;
}

```

OUTPUT:



The screenshot shows the OnlineGDB interface. The left sidebar contains navigation links: 'exp 26', 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main editor displays the following C++ code:

```
1 #include <stdio.h>
2
3 int gcd(int a,int b)
4 {
5     while(b!=0)
6     {
7         int t=b;
8         b=a%b;
9         a=t;
10    }
11    return a;
12 }
13
14 int main()
15 {
16     int n=5590;
17     int plaintext=61; // Shows a factor with n
18     int factor=gcd(plaintext,n);
19     printf("Common factor found = %d\n",factor);
20     printf("Other factor = %d\n",n/factor);
21 }
22
```

The output window at the bottom shows the execution results:

```
Common factor found = 59
Other factor = 61
...Program finished with exit code 0
Press ENTER to exit console.
```

EXPERIMENT 26:

CODE:

```
#include <stdio.h>

long long modExp(long long base, long long exp, long long mod)
{
    long long result = 1;
    while(exp > 0)
    {
        if(exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
```

```

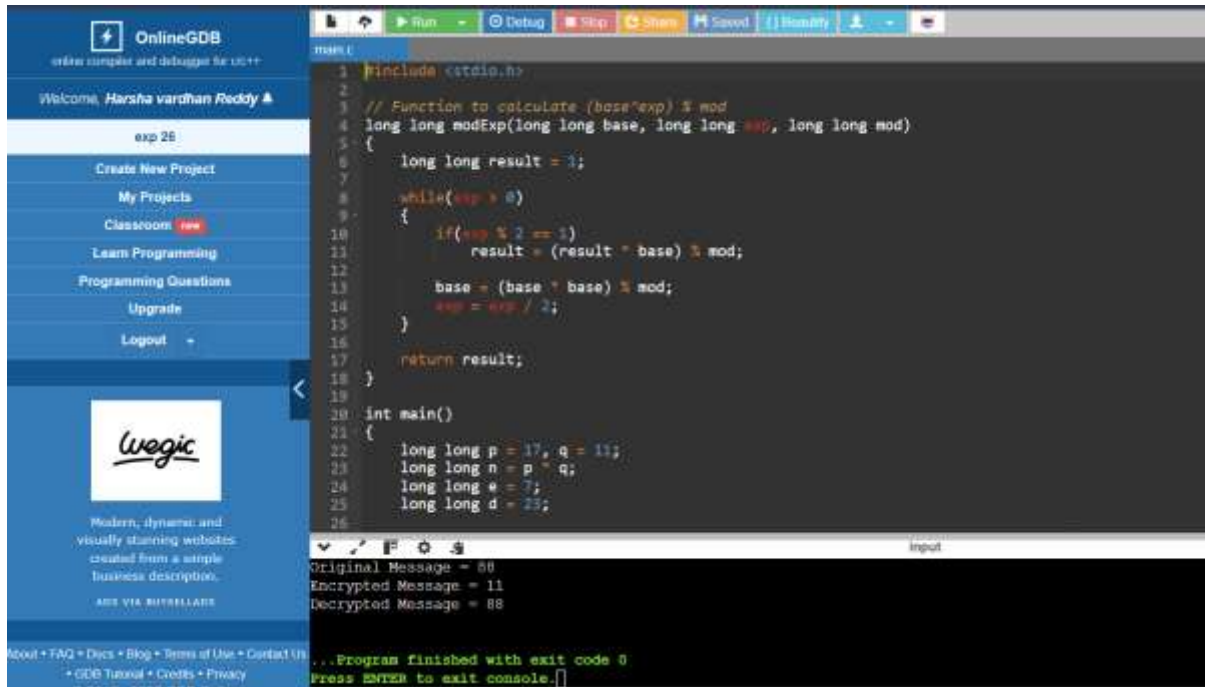
        exp = exp / 2;
    }

    return result;
}

int main()
{
    long long p = 17, q = 11;
    long long n = p * q;
    long long e = 7;
    long long d = 23;
    long long message = 88;
    long long cipher = modExp(message, e, n);
    long long plain = modExp(cipher, d, n);
    printf("Public Key (e,n): (%lld,%lld)\n", e, n);
    printf("Private Key d = %lld\n", d);
    printf("Original Message = %lld\n", message);
    printf("Encrypted Message = %lld\n", cipher);
    printf("Decrypted Message = %lld\n", plain);
    return 0;
}

```

OUTPUT:



```
1 #include <stdio.h>
2
3 // Function to calculate (base^exp) % mod
4 long long modExp(long long base, long long exp, long long mod)
5 {
6     long long result = 1;
7     while(exp > 0)
8     {
9         if(exp % 2 == 1)
10            result = (result * base) % mod;
11         base = (base * base) % mod;
12         exp = exp / 2;
13     }
14     return result;
15 }
16
17 int main()
18 {
19     long long p = 17, q = 11;
20     long long n = p ^ q;
21     long long e = 7;
22     long long d = 23;
23
24     printf("Original Message = 88\n");
25     printf("Encrypted Message = 11\n");
26     printf("Decrypted Message = 88\n");
27
28     ...Program finished with exit code 0
29     Press ENTER to exit console.
```

EXPERIMENT 27:

CODE:

```
#include <stdio.h>
```

```
long long modExp(long long base,long long exp,long long mod)
```

```
{
```

```
    long long result=1;
```

```
    while(exp>0)
```

```
    {
```

```
        if(exp%2)
```

```
            result=(result*base)%mod;
```

```

        base=(base*base)%mod;
        exp/=2;
    }

    return result;
}

int main()
{
    long long e=17,n=3233;
    char text[]="HELLO";

    printf("Ciphertext:\n");

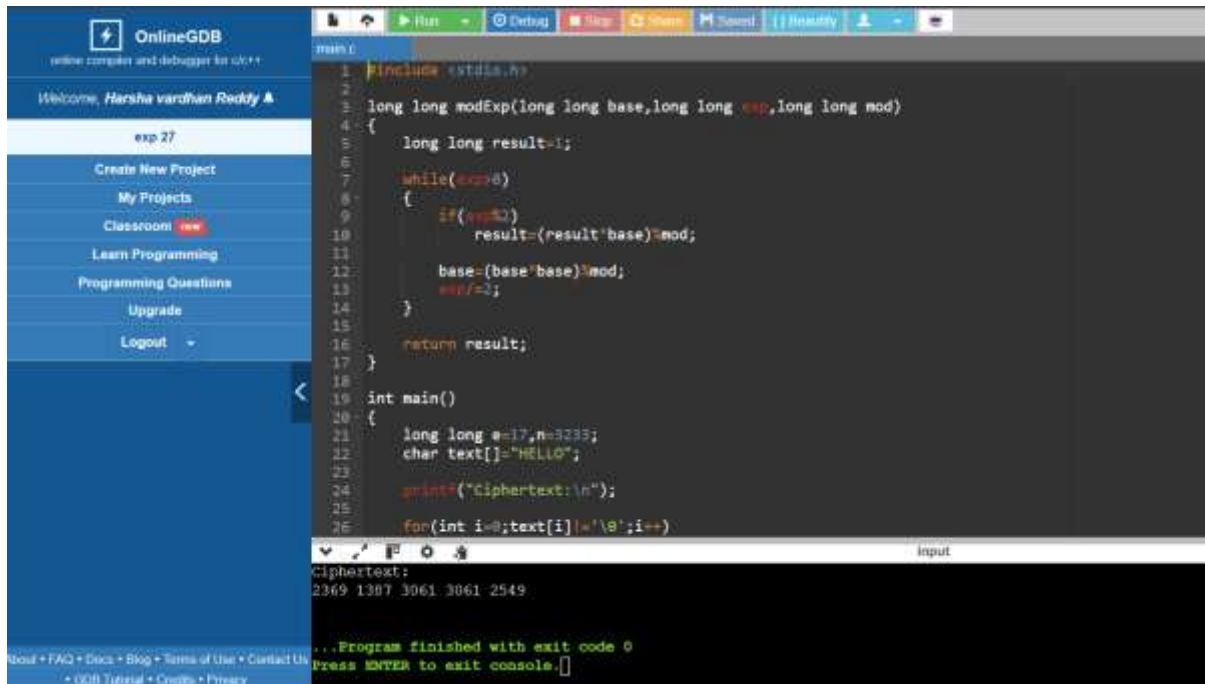
    for(int i=0;text[i]!='\0';i++)
    {
        int m=text[i]-'A';
        printf("%lld ",modExp(m,e,n));
    }

    printf("\n");

    return 0;
}

```

OUTPUT:



The screenshot shows the OnlineGDB interface. The left sidebar contains navigation links: 'Welcome, Harsha vardhan Reddy', 'exp 27', 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main editor displays the following C++ code:

```
1 #include <stdio.h>
2
3 long long modExp(long long base, long long exp, long long mod)
4 {
5     long long result=1;
6
7     while(exp>0)
8     {
9         if(exp%2)
10             result=(result*base)%mod;
11
12         base=(base*base)%mod;
13         exp/=2;
14     }
15
16     return result;
17 }
18
19 int main()
20 {
21     long long a=17, n=3233;
22     char text[]="HELLO";
23     printf("Ciphertext:\n");
24     for(int i=0; text[i]!='\0'; i++)
25         printf("%d ", modExp(a, text[i]-'A', 256));
26 }
```

The output console shows the following text:

```
Ciphertext:
2369 1307 3061 3061 2549

...Program finished with exit code 0
Press ENTER to exit console.
```

EXPERIMENT 28:

CODE:

```
#include <stdio.h>
```

```
long long power(long long a, long long b, long long mod)
```

```
{
```

```
    long long result=1;
```

```

while(b>0)
{
    if(b%2)
        result=(result*a)%mod;

    a=(a*a)%mod;
    b/=2;
}

return result;
}

int main()
{
    long long q=23;
    long long a=5;

    long long xa=6;
    long long xb=15;

    long long Ya=power(a,xa,q);
    long long Yb=power(a,xb,q);

    long long Ka=power(Yb,xa,q);
    long long Kb=power(Ya,xb,q);

    printf("Alice Public Value = %lld\n",Ya);
    printf("Bob Public Value = %lld\n",Yb);

```



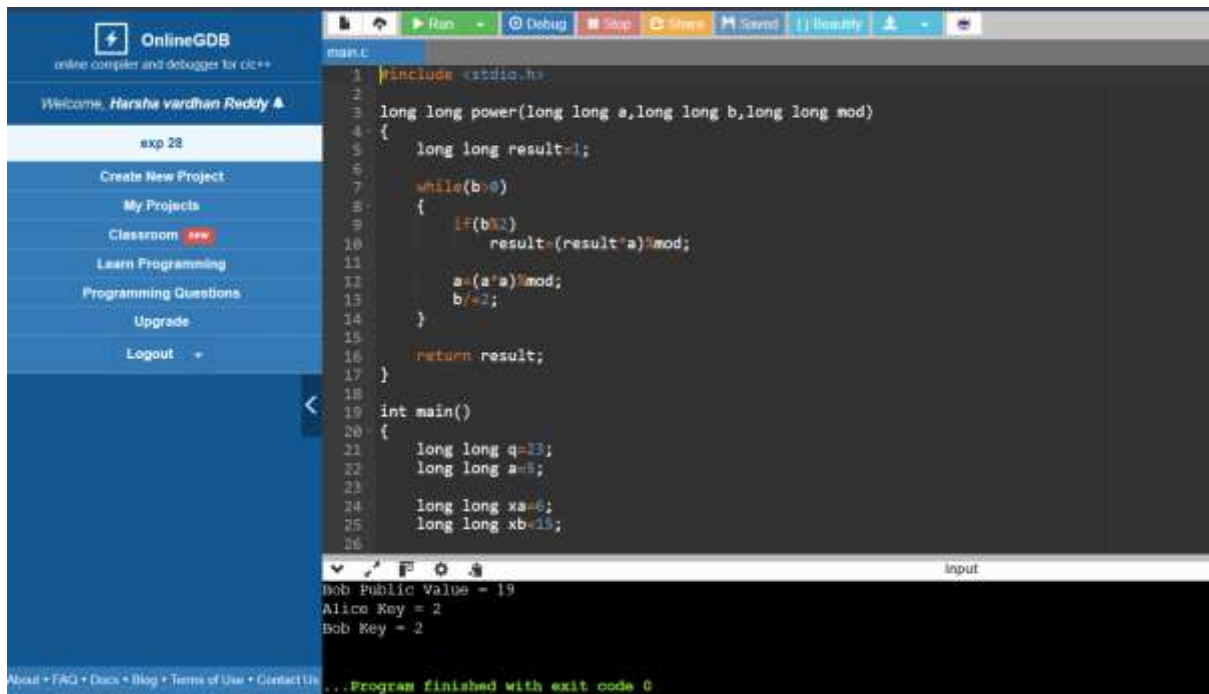
```
printf("Alice Key = %lld\n",Ka);
```

```
printf("Bob Key = %lld\n",Kb);
```

```
return 0;
```

```
}
```

OUTPUT:



```
1 #include <stdio.h>
2
3 long long power(long long a,long long b,long long mod)
4 {
5     long long result=1;
6
7     while(b>0)
8     {
9         if(b%2)
10            result=(result*a)%mod;
11
12         a=(a*a)%mod;
13         b/=2;
14     }
15
16     return result;
17 }
18
19 int main()
20 {
21     long long q=13;
22     long long p=5;
23
24     long long xa=6;
25     long long xb=15;
26
27     Bob Public Value = 19
28     Alice Key = 2
29     Bob Key = 2
30
31     ...Program finished with exit code 0
```

EXPERIMENT 29:

CODE:

```
#include <stdio.h>

int main()
{
    int blockSize = 1024;
    int laneSize = 64;

    int totalLanes = 1600 / laneSize;
    int rateLanes = blockSize / laneSize;
    int capacityLanes = totalLanes - rateLanes;

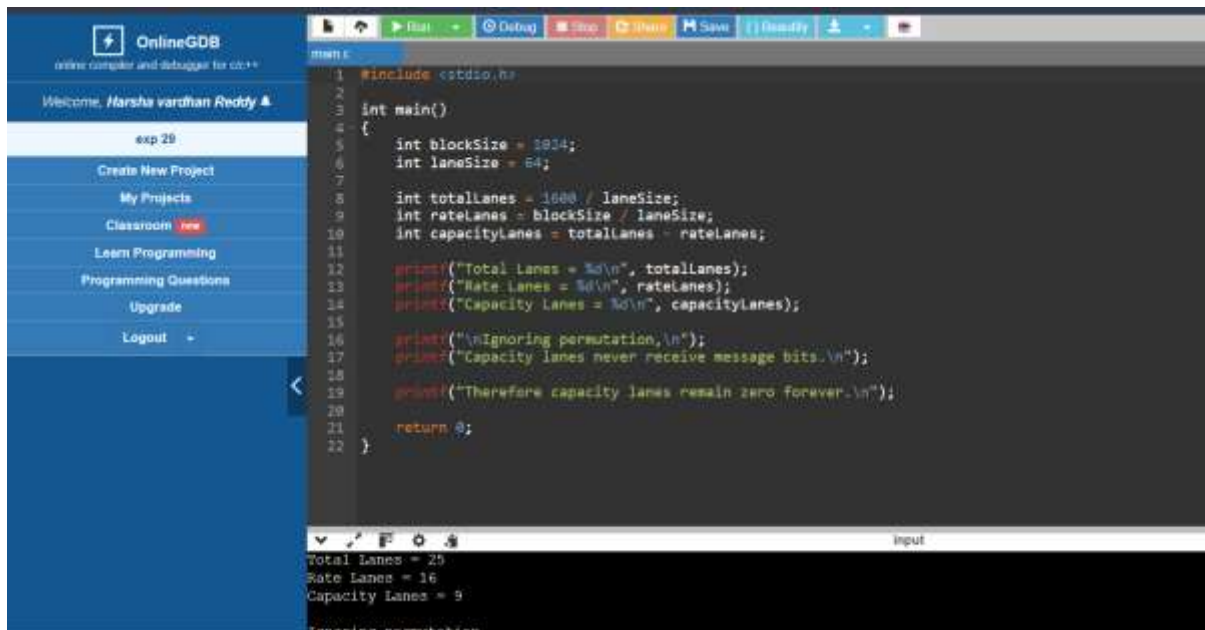
    printf("Total Lanes = %d\n", totalLanes);
    printf("Rate Lanes = %d\n", rateLanes);
    printf("Capacity Lanes = %d\n", capacityLanes);

    printf("\nIgnoring permutation,\n");
    printf("Capacity lanes never receive message bits.\n");

    printf("Therefore capacity lanes remain zero forever.\n");

    return 0;
}
```

OUTPUT:



The screenshot shows the OnlineGDB interface. On the left is a sidebar with navigation links: 'Welcome, Harsha varshan Reddy', 'exp 28', 'Create New Project', 'My Projects', 'Classroom' (highlighted with a red 'new' tag), 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main area displays a C++ program. The code defines constants for blockSize (1024) and laneSize (64), calculates totalLanes (1600 / laneSize), rateLanes (blockSize / laneSize), and capacityLanes (totalLanes - rateLanes). It then prints these values and some explanatory text. The output at the bottom shows the calculated values: Total Lanes = 25, Rate Lanes = 16, and Capacity Lanes = 9.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int blockSize = 1024;
6     int laneSize = 64;
7
8     int totalLanes = 1600 / laneSize;
9     int rateLanes = blockSize / laneSize;
10    int capacityLanes = totalLanes - rateLanes;
11
12    printf("Total Lanes = %d\n", totalLanes);
13    printf("Rate Lanes = %d\n", rateLanes);
14    printf("Capacity Lanes = %d\n", capacityLanes);
15
16    printf("\nIgnoring permutation.\n");
17    printf("Capacity lanes never receive message bits.\n");
18
19    printf("Therefore capacity lanes remain zero forever.\n");
20
21    return 0;
22 }
```

Input

Total Lanes = 25
Rate Lanes = 16
Capacity Lanes = 9
Ignoring permutation

EXPERIMENT 30:

CODE:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    unsigned char X = 0x5A;
```

```
    unsigned char T = 0xA3;
```

```
    unsigned char secondBlock = X ^ T;
```

```
    printf("Message Block X = %02X\n", X);
```

```
    printf("MAC(X) = %02X\n", T);
```

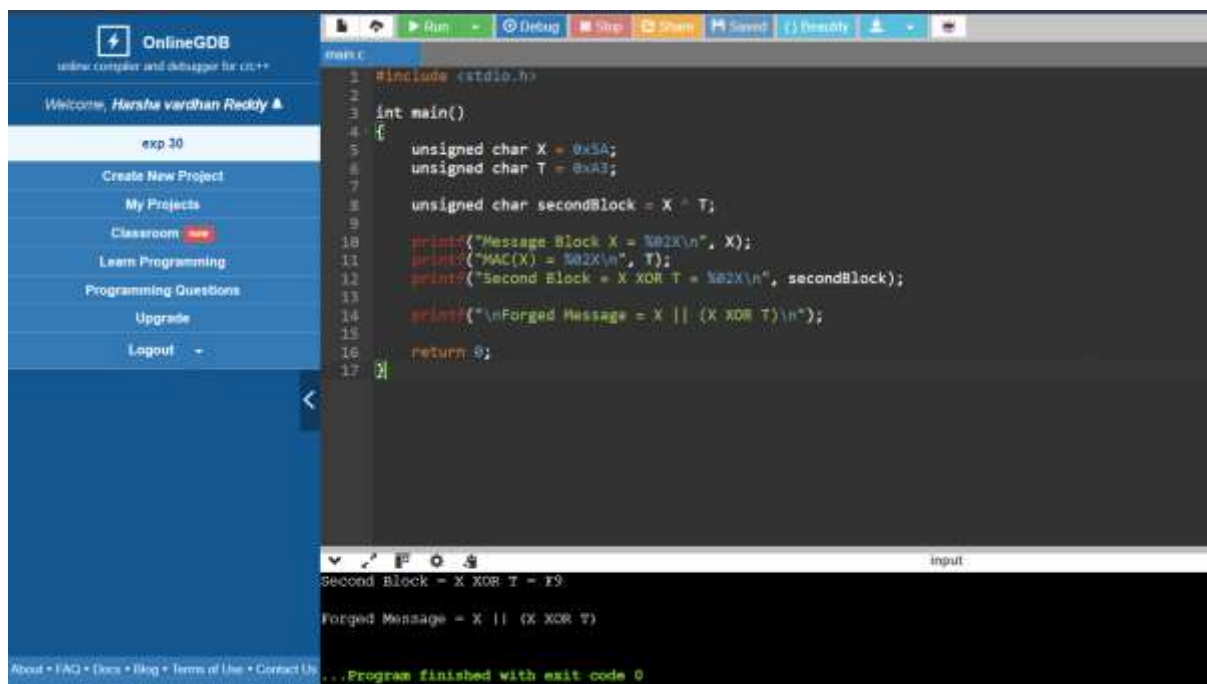
```
    printf("Second Block = X XOR T = %02X\n", secondBlock);
```

```
printf("\nForged Message = X || (X XOR T)\n");
```

```
return 0;
```

```
}
```

OUTPUT:



The screenshot shows the OnlineGDB interface. The left sidebar contains the user's name 'Harshe vardhan Reddy' and a list of navigation links: 'exp 30', 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main editor displays a C program with the following code:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     unsigned char X = 0x5A;
6     unsigned char T = 0xA3;
7     unsigned char secondBlock = X ^ T;
8
9     printf("Message Block X = %02X\n", X);
10    printf("MAC(X) = %02X\n", T);
11    printf("Second Block = X XOR T = %02X\n", secondBlock);
12
13    printf("\nForged Message = X || (X XOR T)\n");
14
15    return 0;
16 }
```

The output window at the bottom shows the execution results:

```
Second Block = X XOR T = F9
Forged Message = X || (X XOR T)
...Program finished with exit code 0
```

EXPERIMENT 31:

CODE:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
uint64_t leftShift64(uint64_t x)
```

```
{
```

```
    return x << 1;
```

```
}
```

```
uint64_t generateK1_64(uint64_t L)
```

```
{
```

```
    uint64_t K1 = leftShift64(L);
```

```
    if (L & 0x8000000000000000ULL)
```

```
        K1 ^= 0x1B;
```

```
    return K1;
```

```
}
```

```
uint64_t generateK2_64(uint64_t K1)
```

```
{
```

```
    uint64_t K2 = leftShift64(K1);
```

```
    if (K1 & 0x8000000000000000ULL)
```

```
        K2 ^= 0x1B;
```

```
    return K2;
```

```
}
```

```
int main()
```

```
{
```

```
    uint64_t L, K1, K2;
```

```
    printf("Enter L in hexadecimal: ");
```

```
    scanf("%llx", &L);
```

```
    K1 = generateK1_64(L);
```

```
K2 = generateK2_64(K1);
```

```
printf("L = %016lX\n", L);
```

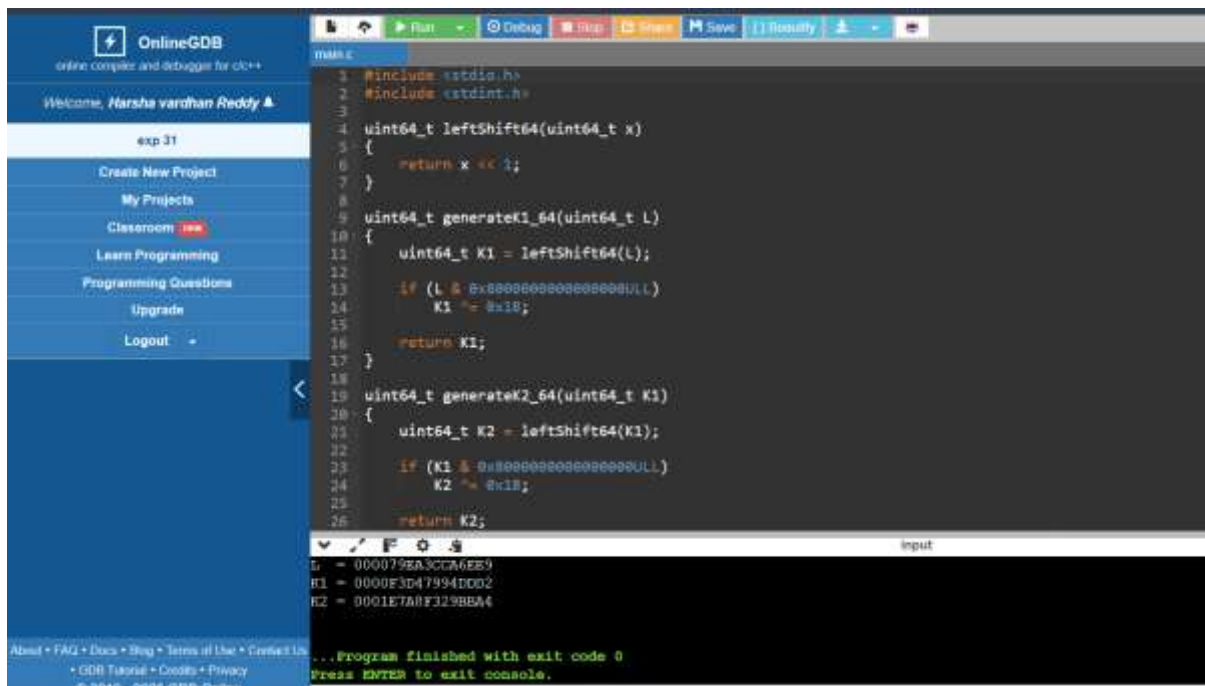
```
printf("K1 = %016lX\n", K1);
```

```
printf("K2 = %016lX\n", K2);
```

```
return 0;
```

```
}
```

OUTPUT:



The screenshot shows the OnlineGDB interface. On the left is a sidebar with navigation links. The main area displays a C program with functions `leftShift64`, `generateK1_64`, and `generateK2_64`. The program takes a value `L` and generates two 64-bit keys, `K1` and `K2`, based on `L`. The output at the bottom shows the hexadecimal values of `L`, `K1`, and `K2`.

```
OnlineGDB
online compiler and debugger for c/c++

Welcome, Harsha vardhan Reddy ▲

exp 31
Create New Project
My Projects
Classroom
Learn Programming
Programming Questions
Upgrade
Logout

main.c
1 #include <stdio.h>
2 #include <stdint.h>
3
4 uint64_t leftShift64(uint64_t x)
5 {
6     return x << 1;
7 }
8
9 uint64_t generateK1_64(uint64_t L)
10 {
11     uint64_t K1 = leftShift64(L);
12     if (L & 0x8000000000000000ULL)
13         K1 ^= 0x1B;
14     return K1;
15 }
16
17 uint64_t generateK2_64(uint64_t K1)
18 {
19     uint64_t K2 = leftShift64(K1);
20     if (K1 & 0x8000000000000000ULL)
21         K2 ^= 0x1B;
22     return K2;
23 }
24
25
26
input
L = 000079EA3C6A6EE9
K1 = 0000F3D47994DD02
K2 = 0001E7A8F329BEA4

...Program finished with exit code 0
Press ENTER to exit console.
```

EXPERIMENT 32:

CODE:

```
#include <stdio.h>

int main()
{
    int messageHash;
    int k1, k2;
    int r1, s1, r2, s2;

    printf("Enter message hash: ");
    scanf("%d", &messageHash);

    printf("Enter k for first signature: ");
    scanf("%d", &k1);

    printf("Enter k for second signature: ");
    scanf("%d", &k2);

    /* Simplified DSA demonstration */
    r1 = (k1 * 7) % 23;
    s1 = (messageHash + k1) % 23;

    r2 = (k2 * 7) % 23;
    s2 = (messageHash + k2) % 23;

    printf("\nFirst Signature : (%d, %d)", r1, s1);
    printf("\nSecond Signature: (%d, %d)", r2, s2);

    if (r1 != r2 || s1 != s2)
```

```

        printf("\n\nSignatures are different.");
    else

        printf("\n\nSignatures are same.");

    printf("\nReason: Different k values were used.\n");

    return 0;
}

```

OUTPUT:

The screenshot shows the OnlineGDB interface with a C program. The code is as follows:

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int messageHash;
6      int k1, k2;
7      int r1, s1, r2, s2;
8
9      printf("Enter message hash: ");
10     scanf("%d", &messageHash);
11
12     printf("Enter k for first signature: ");
13     scanf("%d", &k1);
14
15     printf("Enter k for second signature: ");
16     scanf("%d", &k2);
17
18     /* Simplified DSA demonstration */
19     r1 = (k1 * 7) % 23;
20     s1 = (messageHash + k1) % 23;
21
22     r2 = (k2 * 7) % 23;
23     s2 = (messageHash + k2) % 23;
24
25     printf("\nFirst Signature : (%d, %d)", r1, s1);
26     printf("\nSecond Signature: (%d, %d)", r2, s2);

```

The output of the program is:

```

Signatures are different.
Reason: Different k values were used.

...Program finished with exit code 0
Press ENTER to exit console

```


EXPERIMENT 33:

CODE:

```
#include <stdio.h>

#include <stdint.h>

/* Simplified DES-style demonstration */
uint64_t encrypt(uint64_t data, uint64_t key)
{
    int i;

    for (i = 0; i < 16; i++)
    {
        data = ((data << 3) | (data >> 61));
        data ^= key + i;
    }

    return data;
}

uint64_t decrypt(uint64_t data, uint64_t key)
{
    int i;

    for (i = 15; i >= 0; i--)
    {
        data ^= key + i;
        data = ((data >> 3) | (data << 61));
    }
}
```

```
    return data;
}

int main()
{
    uint64_t plaintext, key;
    uint64_t ciphertext, recovered;

    printf("Enter plaintext (hex): ");
    scanf("%llx", &plaintext);

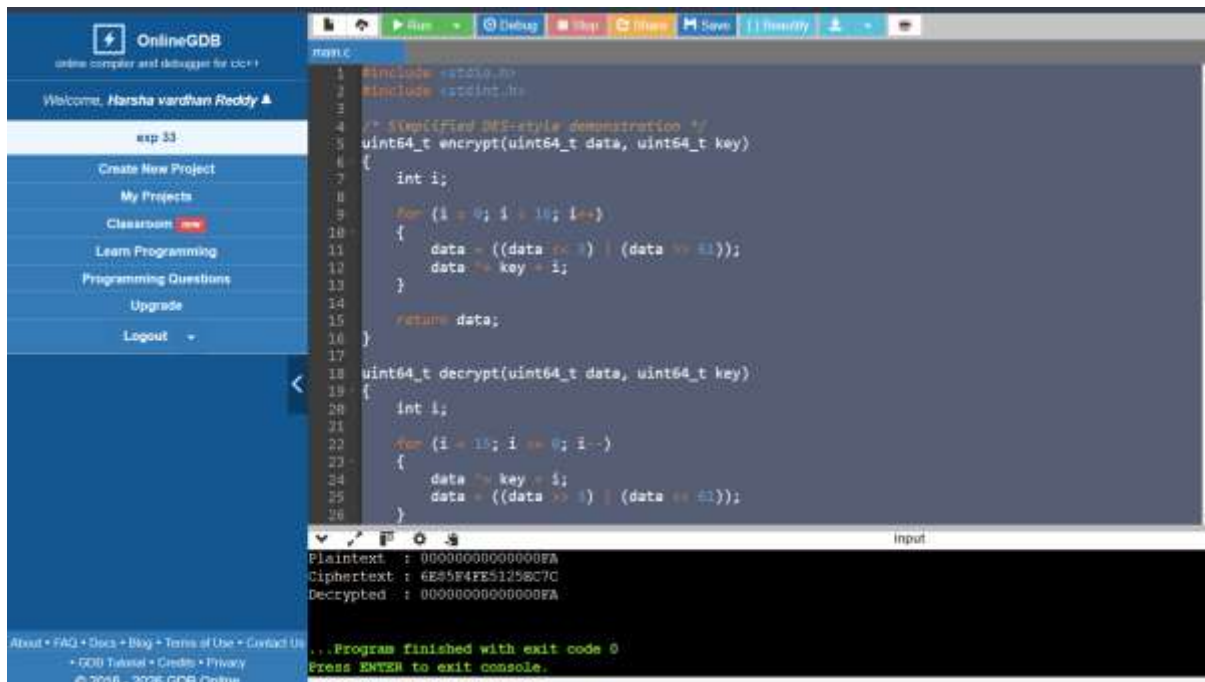
    printf("Enter key (hex): ");
    scanf("%llx", &key);

    ciphertext = encrypt(plaintext, key);
    recovered = decrypt(ciphertext, key);

    printf("\nPlaintext : %016lX", plaintext);
    printf("\nCiphertext : %016lX", ciphertext);
    printf("\nDecrypted : %016lX\n", recovered);

    return 0;
}
```

OUTPUT:



```
OnlineGDB
online compiler and debugger for c/c++

Welcome, Harsha vardhan Reddy ▲

exp 33

Create New Project
My Projects
Classroom new
Learn Programming
Programming Questions
Upgrade
Logout ▼

main.c
1 #include <stdio.h>
2 #include <stdint.h>
3
4 /* Simplified XOR-style demonstration */
5 uint64_t encrypt(uint64_t data, uint64_t key)
6 {
7     int i;
8     for (i = 0; i < 16; i++)
9     {
10         data = ((data << i) | (data >> i));
11         data ^= key + i;
12     }
13     return data;
14 }
15
16
17
18 uint64_t decrypt(uint64_t data, uint64_t key)
19 {
20     int i;
21     for (i = 15; i >= 0; i--)
22     {
23         data ^= key + i;
24         data = ((data >> i) | (data << i));
25     }
26 }
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

Input
Plaintext : 00000000000000FA
Ciphertext : 6E85F4FE51258C7C
Decrypted : 00000000000000FA

...Program finished with exit code 0
Press ENTER to exit console.
```

EXPERIMENT 34:

CODE:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

int encrypt(int block, int key)
{
    return block ^ key;
}

/* ECB */
void ECB(char *plain, int key)
{
    int i;

    printf("ECB Ciphertext: ");

    for (i = 0; plain[i] != '\0'; i++)
        printf("%02X ", encrypt(plain[i], key));

    printf("\n");
}

/* CBC */
void CBC(char *plain, int key, int IV)
{
    int i;
```

```

int previous = IV;

printf("CBC Ciphertext: ");

for (i = 0; plain[i] != '\0'; i++)
{
    int cipher = encrypt(plain[i] ^ previous, key);

    printf("%02X ", cipher);
    previous = cipher;
}

printf("\n");
}

/* CFB */
void CFB(char *plain, int key, int IV)
{
    int i;
    int previous = IV;

    printf("CFB Ciphertext: ");

    for (i = 0; plain[i] != '\0'; i++)
    {
        int stream = encrypt(previous, key);
        int cipher = plain[i] ^ stream;

        printf("%02X ", cipher);
        previous = cipher;
    }
}

```

```
}

printf("\n");
}

int main()
{
    char plaintext[MAX];
    int key, IV;

    printf("Enter plaintext: ");
    fgets(plaintext, MAX, stdin);

    plaintext[strcspn(plaintext, "\n")] = '\0';

    printf("Enter key: ");
    scanf("%d", &key);

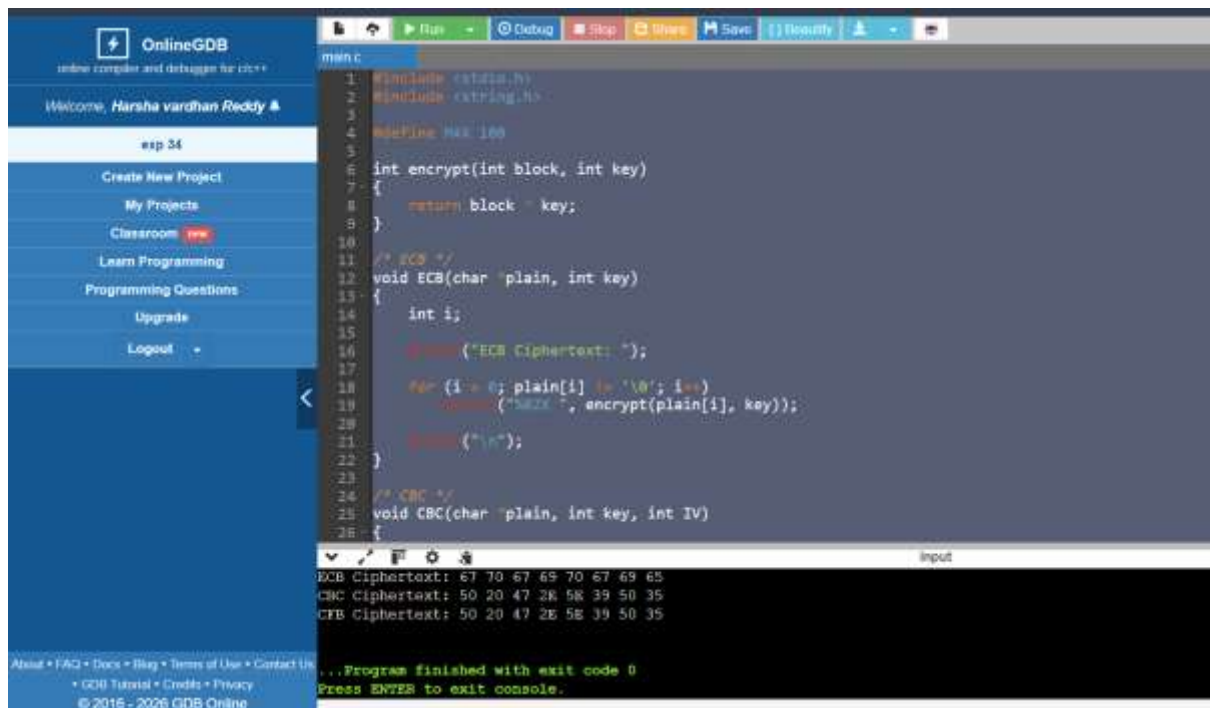
    printf("Enter IV: ");
    scanf("%d", &IV);

    printf("\n");

    ECB(plaintext, key);
    CBC(plaintext, key, IV);
    CFB(plaintext, key, IV);

    return 0;
}
```

OUTPUT:



The screenshot shows the OnlineGDB interface. On the left is a sidebar with the user's name 'Harsha vardhan Reddy' and a menu with options like 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main area displays a C program named 'main.c' with the following code:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX 100
5
6 int encrypt(int block, int key)
7 {
8     return block ^ key;
9 }
10
11 /* ECB */
12 void ECB(char plain, int key)
13 {
14     int i;
15
16     printf("ECB Ciphertext: ");
17
18     for (i = 0; plain[i] != '\0'; i++)
19         printf("%2x ", encrypt(plain[i], key));
20
21     printf("\n");
22 }
23
24 /* CBC */
25 void CBC(char plain, int key, int IV)
26 {
```

Below the code editor, the console output is shown:

```
ECB Ciphertext: 67 70 67 69 70 67 69 65
CBC Ciphertext: 50 20 47 28 58 39 50 35
CFB Ciphertext: 50 20 47 2E 5E 39 50 35
```

At the bottom, a message states: "...Program finished with exit code 0. Press ENTER to exit console."

EXPERIMENT 35:

CODE:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>


int main()
{
    char plaintext[100];
    int key[100];
    int n, i;
    char ciphertext[100];
    char decrypted[100];


    printf("Enter plaintext: ");
    fgets(plaintext, sizeof(plaintext), stdin);

    plaintext[strcspn(plaintext, "\n")] = '\0';

    n = strlen(plaintext);

    printf("Enter %d key values (0-25):\n", n);

    for (i = 0; i < n; i++)
        scanf("%d", &key[i]);


    /* Encryption */
    for (i = 0; i < n; i++)
    {
        if (isalpha(plaintext[i]))
```



```

{
    int p = toupper(plaintext[i]) - 'A';

    ciphertext[i] =
        ((p + key[i]) % 26) + 'A';
}
else
{
    ciphertext[i] = plaintext[i];
}
}

```

```

ciphertext[n] = '\0';

```

```

/* Decryption */

```

```

for (i = 0; i < n; i++)
{
    if (isalpha(ciphertext[i]))
    {
        int c = ciphertext[i] - 'A';

        decrypted[i] =
            ((c - key[i] + 26) % 26) + 'A';
    }
    else
    {
        decrypted[i] = ciphertext[i];
    }
}

```

```
decrypted[n] = '\0';
```

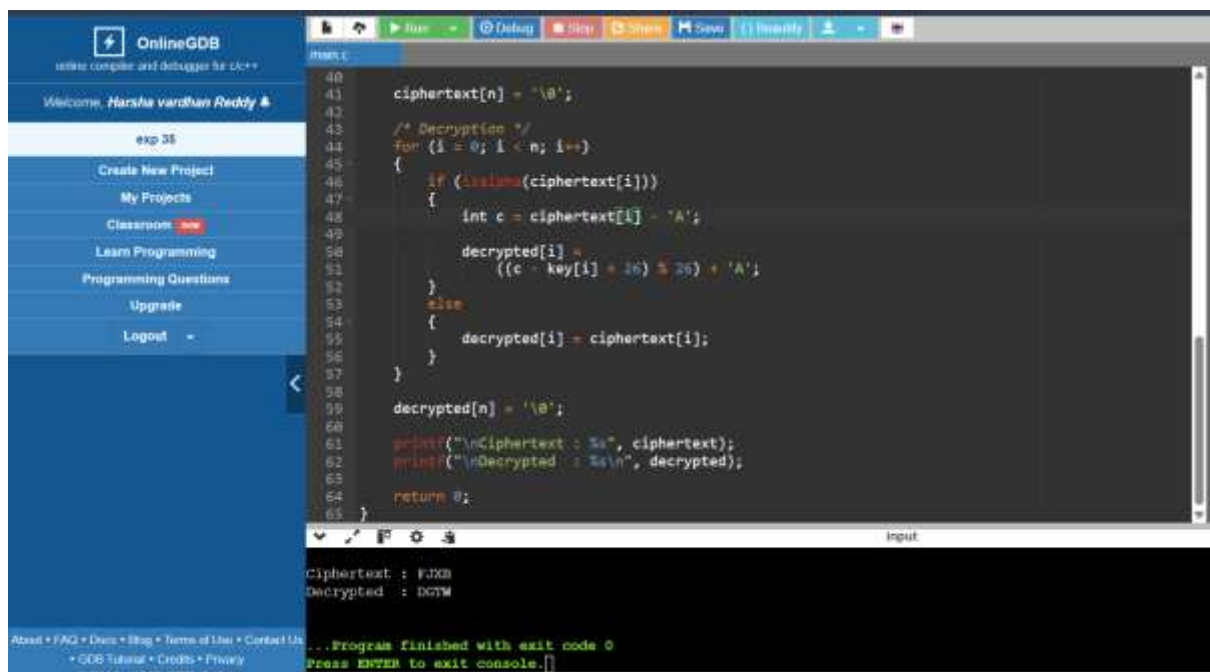
```
printf("\nCiphertext : %s", ciphertext);
```

```
printf("\nDecrypted : %s\n", decrypted);
```

```
return 0;
```

```
}
```

OUTPUT:



The screenshot displays the OnlineGDB interface. On the left is a sidebar with navigation links. The main area shows a C++ program for decrypting a Caesar cipher. The code defines a key 'KEY' as 'QWERTYUIOPASDFGHJKLZXCVBNM', reads a ciphertext 'FJXB' from the user, and prints the decrypted text 'DGTW'. The output console at the bottom shows the program's execution results.

```
OnlineGDB
online compiler and debugger for C/C++

Welcome, Harsha vardhan Reddy

exp 38
Create New Project
My Projects
Classroom
Learn Programming
Programming Questions
Upgrade
Logout

main.c
40 ciphertext[n] = '\0';
41
42 /* Decryption */
43 for (i = 0; i < n; i++)
44 {
45     if (isalpha(ciphertext[i]))
46     {
47         int c = ciphertext[i] - 'A';
48         decrypted[i] =
49             ((c - key[i] + 26) % 26) + 'A';
50     }
51     else
52     {
53         decrypted[i] = ciphertext[i];
54     }
55 }
56
57 decrypted[n] = '\0';
58
59 printf("\nCiphertext : %s", ciphertext);
60 printf("\nDecrypted : %s\n", decrypted);
61
62 return 0;
63 }
64
65
Input
Ciphertext : FJXB
Decrypted : DGTW

...Program finished with exit code 0
Press ENTER to exit console.
```

EXPERIMENT 36:

CODE:

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>


int gcd(int a, int b)
{
    while (b != 0)
    {
        int temp = b;
        b = a % b;
        a = temp;
    }

    return a;
}


int main()
{
    char plaintext[100], ciphertext[100];
    int a, b, i;

    printf("Enter plaintext: ");
    fgets(plaintext, sizeof(plaintext), stdin);

    printf("Enter a: ");
    scanf("%d", &a);

    printf("Enter b: ");
```

```
scanf("%d", &b);
```

```
if (gcd(a, 26) != 1)
```

```
{
```

```
    printf("Invalid a. gcd(a,26) must be 1.\n");
```

```
    return 0;
```

```
}
```

```
for (i = 0; plaintext[i] != '\0'; i++)
```

```
{
```

```
    if (isalpha(plaintext[i]))
```

```
    {
```

```
        int p = toupper(plaintext[i]) - 'A';
```

```
        ciphertext[i] =
```

```
            ((a * p + b) % 26) + 'A';
```

```
    }
```

```
    else
```

```
    {
```

```
        ciphertext[i] = plaintext[i];
```

```
    }
```

```
}
```

```
ciphertext[i] = '\0';
```

```
printf("Ciphertext: %s", ciphertext);
```

```
return 0;
```

```
}
```

OUTPUT:

The screenshot shows the OnlineGDB interface. On the left is a sidebar with navigation links: 'exp 36', 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main area displays a C++ program. The program includes `iostream`, `ctype.h`, and `string.h`. It defines a `gcd` function using a while loop and a `main` function. In `main`, it declares arrays for `plaintext` and `ciphertext`, prompts the user for a plaintext string and two integers `a` and `b`, and then prints the ciphertext. The console output at the bottom shows the user input: 'Enter a: 3', 'Enter b: 4', and 'Ciphertext: MFENL'. The program status indicates it finished with exit code 0.

```
1 #include <iostream.h>
2 #include <ctype.h>
3 #include <string.h>
4
5 int gcd(int a, int b)
6 {
7     while (b != 0)
8     {
9         int temp = b;
10        b = a % b;
11        a = temp;
12    }
13
14    return a;
15 }
16
17 int main()
18 {
19     char plaintext[100], ciphertext[100];
20     int a, b, i;
21
22     cout << "Enter plaintext: ";
23     gets(plaintext, sizeof(plaintext), stdin);
24
25     cout << "Enter a: ";
26     gets("%d", &a);
27
28     cout << "Enter b: ";
29     gets("%d", &b);
30
31     int gcd_val = gcd(a, b);
32
33     for (i = 0; i < strlen(plaintext); i++)
34     {
35         ciphertext[i] = plaintext[i] + gcd_val;
36     }
37
38     ciphertext[i] = '\0';
39
40     cout << "Ciphertext: ";
41     puts(ciphertext);
42
43     return 0;
44 }
```

Enter a: 3
Enter b: 4
Ciphertext: MFENL

...Program finished with exit code 0
Press ENTER to exit console.

EXPERIMENT 37:

CODE:

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>


int main()
{
    char cipher[500];
    char english[] = "ETAOINSHRDLCUMWFGYPBVKJXQZ";

    int freq[26] = {0};
    int order[26];
    int i, j, temp;

    printf("Enter ciphertext: ");
    fgets(cipher, sizeof(cipher), stdin);

    /* Count frequency */
    for (i = 0; cipher[i] != '\0'; i++)
    {
        if (isalpha(cipher[i]))
            freq[toupper(cipher[i]) - 'A']++;
    }

    for (i = 0; i < 26; i++)
        order[i] = i;

    /* Sort by frequency */
    for (i = 0; i < 25; i++)
```

```

{
    for (j = i + 1; j < 26; j++)
    {
        if (freq[order[j]] > freq[order[i]])
        {
            temp = order[i];
            order[i] = order[j];
            order[j] = temp;
        }
    }
}

```

```

printf("\nPossible plaintext:\n");

```

```

for (i = 0; cipher[i] != '\0'; i++)
{
    if (isalpha(cipher[i]))
    {
        int c = toupper(cipher[i]) - 'A';

        for (j = 0; j < 26; j++)
        {
            if (order[j] == c)
            {
                printf("%c", english[j]);
                break;
            }
        }
    }
    else

```

```

    {
        printf("%c", cipher[i]);
    }
}

return 0;
}

```

OUTPUT:

The screenshot shows the OnlineGDB interface. The code editor contains a C program that calculates the Greatest Common Divisor (GCD) of two numbers, a and b, using the Euclidean algorithm. The program prompts the user to enter the plaintext and the key 'a'. The output shows the GCD of 3 and 4 is 1, and the ciphertext is 'MFENL'.

```

1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4
5 int gcd(int a, int b)
6 {
7     while (b != 0)
8     {
9         int temp = b;
10        b = a % b;
11        a = temp;
12    }
13    return a;
14 }
15
16 int main()
17 {
18     char plaintext[100], ciphertext[100];
19     int a, b, i;
20
21     printf("Enter plaintext: ");
22     scanf("%s", plaintext);
23     printf("Enter a: ");
24     scanf("%d", &a);
25 }

```

Enter a: 3
Enter b: 4
Ciphertext: MFENL

...Program finished with exit code 0
Press ENTER to exit console.

EXPERIMENT 38:

CODE:

```
#include <stdio.h>

int inverse(int a)
{
    int i;

    a = (a % 26 + 26) % 26;

    for (i = 1; i < 26; i++)
    {
        if ((a * i) % 26 == 1)
            return i;
    }

    return -1;
}

int main()
{
    int P[2][2], C[2][2], Pinv[2][2], K[2][2];
    int det, invDet;
    int i, j, k;

    printf("Enter plaintext matrix P:\n");

    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++)
            scanf("%d", &P[i][j]);
```

```
printf("Enter ciphertext matrix C:\n");
```

```
for (i = 0; i < 2; i++)
```

```
    for (j = 0; j < 2; j++)
```

```
        scanf("%d", &C[i][j]);
```

```
det = (P[0][0] * P[1][1] -
```

```
        P[0][1] * P[1][0]) % 26;
```

```
if (det < 0)
```

```
    det += 26;
```

```
invDet = inverse(det);
```

```
if (invDet == -1)
```

```
{
```

```
    printf("P has no inverse modulo 26.\n");
```

```
    return 0;
```

```
}
```

```
/* Inverse of P */
```

```
Pinv[0][0] = (P[1][1] * invDet) % 26;
```

```
Pinv[0][1] = (-P[0][1] * invDet) % 26;
```

```
Pinv[1][0] = (-P[1][0] * invDet) % 26;
```

```
Pinv[1][1] = (P[0][0] * invDet) % 26;
```

```
for (i = 0; i < 2; i++)
```

```
    for (j = 0; j < 2; j++)
```

```
        if (Pinv[i][j] < 0)
```

```
Pinv[i][j] += 26;
```

```
/* K = C * P^-1 */
```

```
for (i = 0; i < 2; i++)
```

```
{
```

```
    for (j = 0; j < 2; j++)
```

```
    {
```

```
        K[i][j] = 0;
```

```
        for (k = 0; k < 2; k++)
```

```
            K[i][j] += C[i][k] * Pinv[k][j];
```

```
        K[i][j] %= 26;
```

```
    }
```

```
}
```

```
printf("\nRecovered Key Matrix:\n");
```

```
for (i = 0; i < 2; i++)
```

```
{
```

```
    for (j = 0; j < 2; j++)
```

```
        printf("%d ", K[i][j]);
```

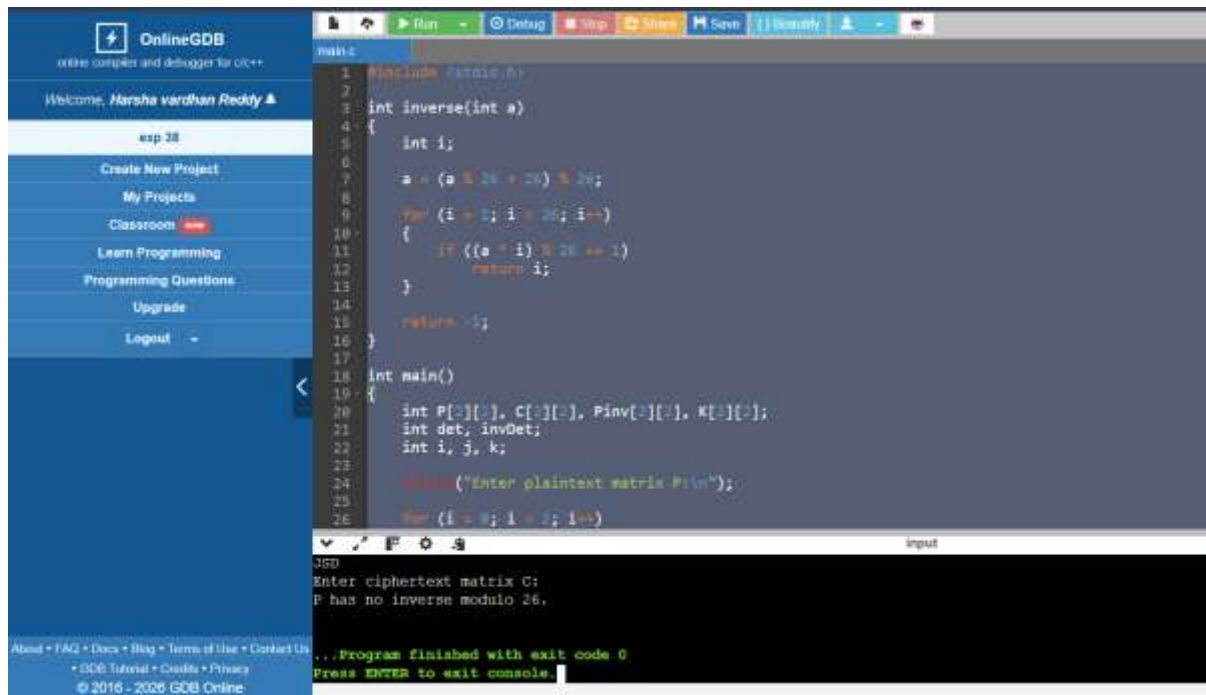
```
    printf("\n");
```

```
}
```

```
return 0;
```

```
}
```

OUTPUT:



The screenshot displays the OnlineGDB interface. On the left is a sidebar with navigation links: 'Welcome, Harsha vardhan Reddy', 'exp 28', 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main area shows a C++ program with the following code:

```
1 #include <stdio.h>
2
3 int inverse(int a)
4 {
5     int i;
6
7     a = (a % 26 + 26) % 26;
8
9     for (i = 1; i < 26; i++)
10     {
11         if ((a * i) % 26 == 1)
12             return i;
13     }
14
15     return -1;
16 }
17
18 int main()
19 {
20     int P[2][2], C[2][2], Pinv[2][2], K[2][2];
21     int det, invDet;
22     int i, j, k;
23
24     printf("Enter plaintext matrix P:\n");
25
26     for (i = 0; i < 2; i++)
```

The console output at the bottom shows the program's execution:

```
350
Enter ciphertext matrix C:
P has no inverse modulo 26.
...Program finished with exit code 0
Press ENTER to exit console.
```

EXPERIMENT 39:

CODE:

```
#include <stdio.h>

#include <ctype.h>

int main()
{
    char cipher[500];
    int freq[26] = {0};
    int i, max = 0, maxIndex = 0;
    int key;

    printf("Enter ciphertext: ");
    fgets(cipher, sizeof(cipher), stdin);

    /* Count frequency */
    for (i = 0; cipher[i] != '\0'; i++)
    {
        if (isalpha(cipher[i]))
            freq[toupper(cipher[i]) - 'A']++;
    }

    /* Find most frequent letter */
    for (i = 0; i < 26; i++)
    {
        if (freq[i] > max)
        {
            max = freq[i];
            maxIndex = i;
        }
    }
}
```

```

}

/* Assume most frequent letter is E */
key = (maxIndex - 4 + 26) % 26;

printf("\nMost frequent letter: %c\n",
       maxIndex + 'A');

printf("Possible key: %d\n", key);

printf("Possible plaintext: ");

for (i = 0; cipher[i] != '\0'; i++)
{
    if (isalpha(cipher[i]))
    {
        int c = toupper(cipher[i]) - 'A';
        int p = (c - key + 26) % 26;

        printf("%c", p + 'A');
    }
    else
    {
        printf("%c", cipher[i]);
    }
}

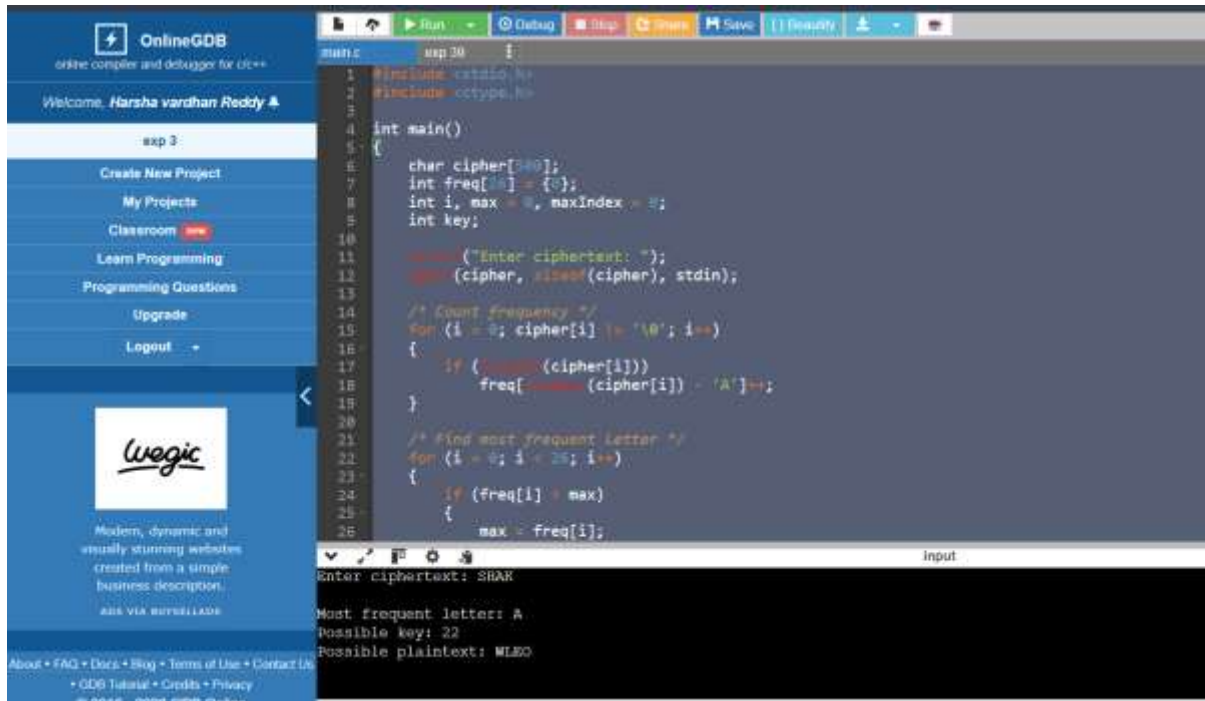
printf("\n");

return 0;

```

}

OUTPUT:



```
1 #include <stdio.h>
2 #include <ctype.h>
3
4 int main()
5 {
6     char cipher[540];
7     int freq[26] = {0};
8     int i, max = 0, maxIndex = 0;
9     int key;
10
11     printf("Enter ciphertext: ");
12     scanf("%s", cipher);
13
14     /* Count frequency */
15     for (i = 0; cipher[i] != '\0'; i++)
16     {
17         if (isalpha(cipher[i]))
18             freq[cipher[i] - 'A']++;
19     }
20
21     /* Find most frequent letter */
22     for (i = 0; i < 26; i++)
23     {
24         if (freq[i] > max)
25             max = freq[i];
26     }
```

Enter ciphertext: SBAR

Most frequent letter: A

Possible key: 22

Possible plaintext: MLEO

EXPERIMENT 40:

CODE:

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#include <string.h>
```

```
void decrypt(char *cipher, int shift, char *plain)
```

```
{
```

```
    int i;
```

```
    for (i = 0; cipher[i] != '\0'; i++)
```

```
{
```

```

        if (isalpha(cipher[i]))
        {
            int c = toupper(cipher[i]) - 'A';
            plain[i] = ((c - shift + 26) % 26) + 'A';
        }
        else
        {
            plain[i] = cipher[i];
        }
    }

    plain[i] = '\0';
}

```

```

int main()
{
    char cipher[500];
    char plain[500];
    int freq[26] = {0};
    int i, j, max, maxIndex;

    printf("Enter ciphertext: ");
    fgets(cipher, sizeof(cipher), stdin);

    /* Count frequency */
    for (i = 0; cipher[i] != '\0'; i++)
    {
        if (isalpha(cipher[i]))
            freq[toupper(cipher[i]) - 'A']++;
    }
}

```



```

/* Find most frequent letter */

max = 0;
maxIndex = 0;

for (i = 0; i < 26; i++)
{
    if (freq[i] > max)
    {
        max = freq[i];
        maxIndex = i;
    }
}

printf("\nTop 10 possible plaintexts:\n\n");

/*
    Try 10 shifts near the probable shift.
*/
for (j = 0; j < 10; j++)
{
    int key = (maxIndex - 4 + j + 26) % 26;

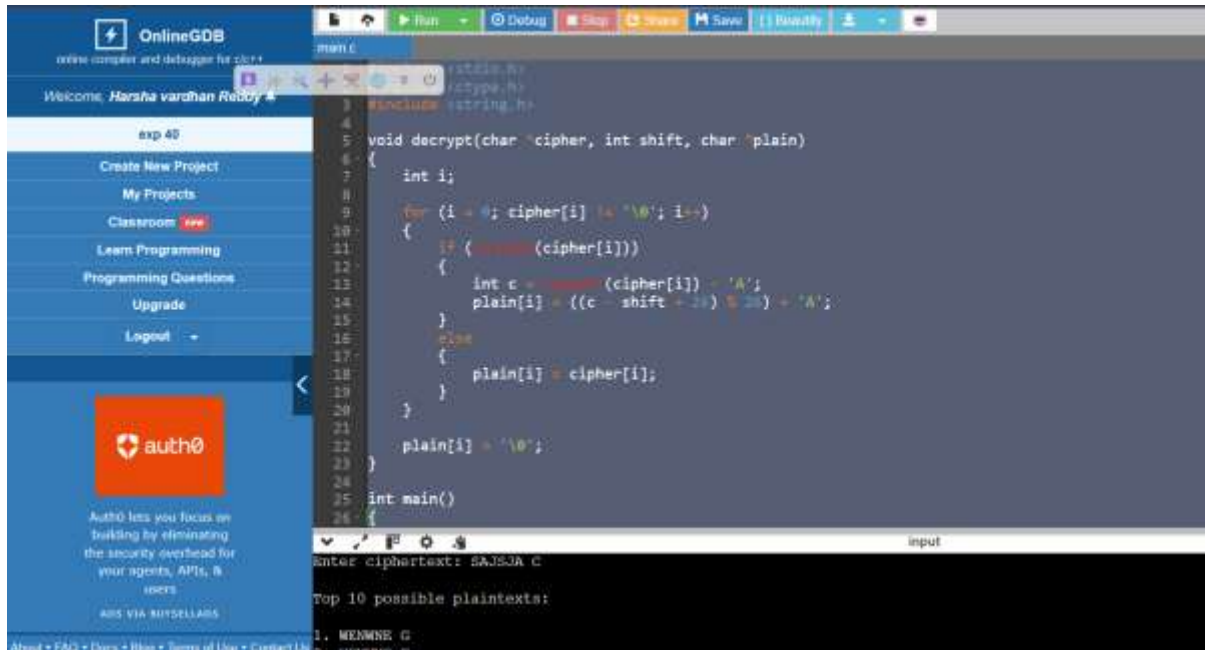
    decrypt(cipher, key, plain);

    printf("%d. %s", j + 1, plain);
}

return 0;
}

```

OUTPUT:



The screenshot shows the OnlineGDB interface. On the left, there's a sidebar with navigation links: 'exp 40', 'Create New Project', 'My Projects', 'Classroom', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. Below this is an 'auth0' logo and a description: 'Auth0 lets you focus on building by eliminating the security overhead for your agents, APIs, & users. ADD VIA BUYSELLARS'. At the bottom of the sidebar are links: 'About', 'FAQ', 'Docs', 'Blog', 'Terms of Use', and 'Contact Us'.

The main area displays a C program for decrypting a Caesar cipher. The code is as follows:

```
1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4
5 void decrypt(char *cipher, int shift, char *plain)
6 {
7     int i;
8     for (i = 0; cipher[i] != '\0'; i++)
9     {
10         if (isalpha(cipher[i]))
11         {
12             int c = isupper(cipher[i]) ? 'A' : 'a';
13             plain[i] = ((c - shift + 26) % 26) + 'A';
14         }
15         else
16         {
17             plain[i] = cipher[i];
18         }
19     }
20     plain[i] = '\0';
21 }
22
23 int main()
24 {
25     char cipher[100];
26     char plain[100];
27     int shift;
```

The program prompts the user to enter a ciphertext. The input shown is 'SAJSJA C'. The program then outputs the top 10 possible plaintexts, with the first one being 'MENNNE C'.

```
1. MENNNE C
2. ...
```